

*À ma famille, dont le soutien indéfectible
a été le fondement de chacun de mes pas.*

*À chaque chercheur et étudiant en Tunisie
qui mérite un accès simplifié aux ressources informatiques.*

Remerciements

Je tiens à exprimer ma sincère gratitude envers toutes les personnes qui m'ont soutenue tout au long de ce projet de fin d'études.

Je remercie chaleureusement mon encadrante académique à l'ESPRIT, **Mme Khadija Bessadak**, pour sa disponibilité, ses conseils avisés et son suivi rigoureux tout au long du stage.

J'adresse également mes remerciements à mon encadrant en entreprise chez Huawei Technologies Tunisie, **M. Rached Abdeljaoued**, pour m'avoir accueillie au sein de son équipe, partagé son expertise en infrastructure cloud et mis à ma disposition l'environnement Huawei Cloud Stack (HCS) qui a rendu ce projet possible.

Je remercie toute l'équipe de Huawei Tunisie pour leur accueil chaleureux et les échanges techniques enrichissants qui m'ont permis de mieux appréhender les spécificités de la plateforme HCS.

Enfin, je suis reconnaissante envers l'ensemble du corps enseignant de l'ESPRIT pour la formation de qualité qui m'a préparée à cette expérience, ainsi qu'envers ma famille et mes amis pour leur soutien constant.

Résumé

L'adoption du cloud computing dans les institutions académiques tunisiennes est freinée par l'absence de portails en libre-service pour l'infrastructure nationale Huawei Cloud Stack (HCS) gérée par le Ministère de l'Enseignement Supérieur et de la Recherche Scientifique (MESRS). Les chercheurs et étudiants souhaitant accéder à des machines virtuelles à la demande doivent recourir à des processus administratifs manuels et chronophages, sans visibilité sur l'avancement du provisionnement.

Ce projet présente **VM Marketplace**, une plateforme cloud full-stack en libre-service, conçue et développée lors d'un stage de six mois chez Huawei Technologies Tunisie. La plateforme permet aux utilisateurs authentifiés de configurer, payer et provisionner des machines virtuelles et des clusters Kubernetes directement sur l'infrastructure HCS via une interface web intuitive. Ses fonctionnalités clés comprennent : un assistant de configuration de VM en plusieurs étapes avec suivi en temps réel via Server-Sent Events, un moteur de recommandation IA basé sur le modèle de langage Llama 3.1, un terminal SSH dans le navigateur, une installation automatisée de Netdata pour la supervision, un gestionnaire complet du cycle de vie des clusters k3s, un contrôleur d'autoguérison hub-spoke et des outils de synchronisation entre la base de données et l'état Terraform cloud.

L'architecture repose sur trois services indépendants : un frontend Next.js, un backend FastAPI et un moteur d'inférence IA distinct. Le provisionnement d'infrastructure est géré par Terraform avec le fournisseur huaweicloud/hcs. Les clusters Kubernetes sont initialisés avec k3s (distribution légère de Rancher Labs). Un schéma réseau de type « maître-comme-passerelle-NAT » — combinant `source_dest_check=false` au niveau de l'hyperviseur avec le masquage IP Linux — a été conçu et validé afin de fournir aux nuds workers un accès internet malgré les contraintes de quota d'EIP de l'environnement HCS partagé.

La validation sur l'infrastructure HCS de production a confirmé le provisionnement complet de machines virtuelles et d'un cluster à deux nuds sous k3s v1.31.4+k3s1, avec les deux nuds en statut Ready.

Mots-clés : Cloud computing, Huawei Cloud Stack, Machines virtuelles, Kubernetes, k3s, Infrastructure as Code, Terraform, Recommandation IA, FastAPI, Next.js, Portail libre-service.

Table des matières

Remerciements	ii
Résumé	iii
Liste des abréviations	ix
Introduction Générale	1
1 Contexte Général	3
1.1 Introduction	3
1.2 Présentation de l'Organisme d'Accueil	3
1.2.1 Informations Générales	3
1.2.2 Huawei Cloud Stack (HCS)	3
1.2.3 Contexte du Stage	3
1.3 Contexte du Projet et Problématique	4
1.3.1 Contexte du Projet	4
1.3.2 Problématique	4
1.4 Étude de l'Existant	4
1.4.1 Présentation des Solutions Candidates	4
1.4.2 Analyse Comparative	5
1.5 Critique de l'Existant	5
1.6 Solution Proposée	6
1.6.1 Vue d'Ensemble	6
1.6.2 Architecture Générale	7
1.7 Méthodologie	7
1.7.1 Comparaison des Méthodologies de Développement	7
1.7.2 Méthodologie Retenue : Agile Itératif	7
1.7.3 Planification du Projet	8
1.8 Conclusion	8
2 Analyse et Architecture	9
2.1 Introduction	9
2.2 Identification des Acteurs	9
2.3 Besoins Fonctionnels	9
2.3.1 Module Authentification	9
2.3.2 Module Provisionnement de VM	9
2.3.3 Module Gestion des VM	10

2.3.4	Module Recommandation IA	10
2.3.5	Module Cluster Kubernetes	10
2.4	Besoins Non Fonctionnels	11
2.5	Diagramme de Cas d'Utilisation Global	11
2.6	Diagramme de Classes	12
2.7	Backlog Produit	13
2.8	Architecture du Système	14
2.8.1	Architecture Logique	14
2.8.2	Architecture de Déploiement	15
2.9	Environnement de Travail	15
2.10	Choix Technologiques	16
2.10.1	Frontend : Next.js 14	16
2.10.2	Backend : FastAPI	16
2.10.3	Infrastructure as Code : Terraform	16
2.10.4	Moteur IA : Ollama + Llama 3.1	16
2.10.5	Connectivité SSH : Paramiko	16
2.10.6	Supervision : Netdata	16
2.11	Conclusion	17
3	Conception et Réalisation	18
3.1	Introduction	18
3.2	Sprint 1 — Authentification et Provisionnement de VM	18
3.2.1	Objectif du Sprint	18
3.2.2	Backlog du Sprint	18
3.2.3	Conception de l'Authentification	18
3.2.4	Assistant de Configuration de VM	19
3.2.5	Pipeline de Provisionnement de VM	19
3.2.6	Tableau de Bord	21
3.2.7	Difficultés Rencontrées dans le Sprint 1	21
3.3	Sprint 2 — Recommandation IA, Terminal SSH et Supervision	21
3.3.1	Objectif du Sprint	21
3.3.2	Backlog du Sprint	22
3.3.3	Moteur de Recommandation IA	22
3.3.4	Terminal SSH dans le Navigateur	23
3.3.5	Supervision Netdata	23
3.3.6	Marketplace de VM	24
3.4	Sprint 3 — Clusters k3s, Autoguérison et Synchronisation Cloud	24
3.4.1	Objectif du Sprint	24
3.4.2	Backlog du Sprint	24
3.4.3	Choix de k3s comme Distribution Kubernetes	25
3.4.4	Architecture du Cluster	25
3.4.5	Modèle Terraform du Cluster	26

3.4.6	Séquence d'Initialisation k3s	27
3.4.7	Solution Réseau : Maître comme Passerelle NAT	27
3.4.8	Assistant et Tableau de Bord du Cluster	28
3.4.9	Terminal SSH pour Clusters	29
3.4.10	Contrôleur d'Autoguérison (Self-Healing)	29
3.4.11	Synchronisation et Import depuis le Cloud	30
3.5	Conclusion	31
4	Tests et Validation	32
4.1	Introduction	32
4.2	Environnement de Test	32
4.3	Tests d'Authentification	33
4.4	Tests de Provisionnement de VM	35
4.5	Tests de Recommandation IA	36
4.6	Tests du Terminal SSH	37
4.7	Tests du Cluster k3s	39
4.8	Tests d'Autoguérison et de Synchronisation	41
4.9	Évaluation des Objectifs	41
4.10	Limites Identifiées	43
4.11	Conclusion	43
	Conclusion Générale	44
	Bibliographie / Netographie	48
	A Référence des Endpoints REST	49
	B Modèle Terraform du Cluster	51

Table des figures

1.1	Architecture générale de VM Marketplace	7
1.2	Diagramme de Gantt — planification du projet VM Marketplace ([TODO : ajuster les numéros de semaines selon la date de début réelle du stage]) . . .	8
2.1	Diagramme de cas d'utilisation global — VM Marketplace	12
2.2	Modèle de données principal (entités ORM SQLAlchemy)	13
2.3	Architecture de déploiement de VM Marketplace	15
3.1	Diagramme de séquence — Authentification	19
3.2	Diagramme de séquence — Provisionnement de VM	20
3.3	Flux de données de la recommandation IA	22
3.4	Architecture réseau du cluster k3s sur HCS	26

Liste des tableaux

1.1	Comparaison des solutions cloud en libre-service existantes	5
1.2	Comparaison des méthodologies de développement	7
2.1	Besoins non fonctionnels	11
2.2	Backlog produit	14
2.3	Environnement de développement	15
3.1	Backlog Sprint 1	18
3.2	Backlog Sprint 2	22
3.3	Backlog Sprint 3	24
4.1	Environnement de test	32
4.2	Résultats des tests d'authentification	33
4.3	Résultats des tests de provisionnement de VM	35
4.4	Résultats des tests de recommandation IA	36
4.5	Résultats des tests du terminal SSH	37
4.6	Résultats des tests du cluster k3s	39
4.7	Résultats des tests d'autoguérison et de synchronisation	41
4.8	Évaluation des objectifs	42
A.1	Endpoints REST de VM Marketplace	50

Liste des abréviations

Abréviation	Définition
IA	Intelligence Artificielle
API	Application Programming Interface (Interface de programmation applicative)
CNI	Container Network Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete (Créer, Lire, Mettre à jour, Supprimer)
BD	Base de Données
DNS	Domain Name System (Système de noms de domaine)
ECS	Elastic Cloud Server (Serveur cloud élastique)
EIP	Elastic IP Address (Adresse IP élastique)
HCS	Huawei Cloud Stack
HTTP	Hypertext Transfer Protocol
IaC	Infrastructure as Code (Infrastructure en tant que code)
IAM	Identity and Access Management
IP	Internet Protocol
JWT	JSON Web Token
K8s	Kubernetes
LLM	Large Language Model (Grand modèle de langage)
MESRS	Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
NAT	Network Address Translation (Traduction d'adresses réseau)
ORM	Object-Relational Mapping (Correspondance objet-relationnel)
PFE	Projet de Fin d'Études
REST	Representational State Transfer
SQL	Structured Query Language
SSH	Secure Shell
SSE	Server-Sent Events (Événements envoyés par le serveur)
IHM	Interface Homme-Machine
URL	Uniform Resource Locator
VM	Virtual Machine (Machine Virtuelle)
VPC	Virtual Private Cloud
WS	WebSocket

Introduction Générale

Le cloud computing est devenu le socle de la recherche scientifique et de l'enseignement supérieur modernes. La capacité à provisionner des ressources informatiques à la demande — machines virtuelles, conteneurs et environnements cluster — supprime les obstacles traditionnels liés aux cycles d'acquisition de matériel et permet aux chercheurs d'itérer plus rapidement. En Tunisie, le Ministère de l'Enseignement Supérieur et de la Recherche Scientifique (MESRS) opère une infrastructure cloud nationale Huawei Cloud Stack (HCS) à l'adresse *mesrsccloud.rnu.tn*, mettant des ressources cloud privées à la disposition des institutions académiques à travers le pays. Cependant, l'accès à ces ressources est resté un processus manuel et technique : les utilisateurs doivent soumettre des demandes aux administrateurs, attendre le provisionnement et recevoir leurs identifiants sans aucune visibilité sur le processus et sans autonomie de libre-service.

Mission de stage. Ce projet de fin d'études a été réalisé chez Huawei Technologies Tunisie avec la mission suivante : concevoir et développer une plateforme cloud en libre-service — *VM Marketplace* — permettant aux utilisateurs authentifiés de l'environnement HCS MESRS de configurer, payer, provisionner, superviser et gérer de manière autonome des machines virtuelles et des clusters Kubernetes via une interface web intuitive, sans intervention d'un administrateur pour les opérations courantes.

Problématique. La problématique centrale de ce projet est la suivante : *Comment permettre aux utilisateurs académiques de l'infrastructure cloud HCS du MESRS de provisionner et gérer de façon autonome des machines virtuelles et des clusters Kubernetes via une plateforme conviviale intégrant paiement, recommandation IA, suivi en temps réel et supervision automatisée ?*

Objectifs. Pour répondre à cette problématique, cinq objectifs techniques ont été définis :

1. Implémenter une application web full-stack sécurisée avec authentification utilisateur, assistant de provisionnement de VM en plusieurs étapes, intégration du paiement Stripe et infrastructure as code Terraform sur HCS.
2. Développer un moteur de recommandation IA qui traduit des descriptions textuelles de charge de travail en spécifications de VM optimales à l'aide d'un LLM hébergé localement.
3. Ajouter un terminal SSH dans le navigateur et une installation automatisée de Netdata pour chaque VM provisionnée.
4. Concevoir et implémenter un pipeline de provisionnement de clusters Kubernetes qui initialise des clusters multi-nuds sur HCS avec kubeadm, en incluant une solution réseau permettant aux nuds workers d'accéder à Internet malgré les contraintes de quota EIP.
5. Valider l'ensemble de la plateforme sur l'infrastructure HCS de production.

Structure du rapport. Ce rapport est organisé en quatre chapitres suivis d'une conclusion générale.

- **Chapitre 1 — Contexte général** présente l'organisme d'accueil, situe le projet dans l'écosystème cloud académique du MESRS, étudie les solutions existantes et justifie l'approche proposée et la méthodologie.
- **Chapitre 2 — Analyse et Architecture** identifie les acteurs et les besoins, établit le backlog produit et détaille l'architecture à trois services et les choix technologiques.
- **Chapitre 3 — Conception et Réalisation** documente l'implémentation à travers trois sprints itératifs : provisionnement de VM et authentification, recommandation IA et supervision, et gestion des clusters Kubernetes.
- **Chapitre 4 — Tests et Validation** présente les résultats des tests et évalue la plateforme au regard des objectifs initiaux.

Chapitre 1 Contexte Général

1.1 Introduction

Ce chapitre établit le contexte dans lequel VM Marketplace a été conçue et développée. Nous présentons d’abord l’organisme d’accueil, puis nous situons le projet dans le paysage cloud académique tunisien, définissons la problématique, étudions les solutions existantes, proposons notre réponse et justifions la méthodologie de développement adoptée.

1.2 Présentation de l’Organisme d’Accueil

1.2.1 Informations Générales

Huawei Technologies Co., Ltd. est une multinationale chinoise de technologie fondée en 1987 à Shenzhen, Chine, par Ren Zhengfei. Elle est aujourd’hui l’un des plus grands fournisseurs mondiaux d’infrastructure TIC (Technologies de l’Information et de la Communication) et de dispositifs intelligents, avec des activités dans plus de 170 pays et un chiffre d’affaires dépassant 92 milliards de dollars en 2023 [1].

Huawei Technologies Tunisie est la filiale locale chargée du déploiement des solutions entreprise et gouvernementales de Huawei sur le marché nord-africain. En Tunisie, la société a établi un partenariat stratégique avec le MESRS pour fournir et opérer l’infrastructure nationale **Huawei Cloud Stack (HCS)** au profit des institutions académiques [2].

1.2.2 Huawei Cloud Stack (HCS)

Huawei Cloud Stack est la plateforme cloud on-premises de Huawei, conçue pour les déploiements gouvernementaux et d’entreprise où la souveraineté des données exige que les ressources de calcul restent dans un périmètre physique contrôlé. Le déploiement MESRS, accessible à l’adresse *mesrscloud.rnu.tn*, met à disposition des universités et laboratoires de recherche des instances ECS (Elastic Cloud Server), des réseaux VPC (Virtual Private Cloud), des adresses IP élastiques (EIP) et du stockage objet [3].

HCS expose des API compatibles OpenStack, et Huawei distribue un fournisseur Terraform officiel (*huaweicloud/hcs*) permettant la gestion en infrastructure as code de toutes les ressources cloud [4].

1.2.3 Contexte du Stage

Le stage s’est déroulé au sein de l’équipe Infrastructure Cloud de Huawei Technologies Tunisie. L’équipe est responsable de la maintenance du déploiement HCS et de l’assistance

technique aux institutions académiques. La mission de la stagiaire consistait à concevoir et livrer un portail en libre-service réduisant la charge manuelle des administrateurs tout en donnant de l'autonomie aux utilisateurs finaux.

1.3 Contexte du Projet et Problématique

1.3.1 Contexte du Projet

Les chercheurs et étudiants des universités tunisiennes ont fréquemment besoin de machines virtuelles pour leurs expériences computationnelles, analyses de données et charges de travail containerisées. L'accès à la plateforme HCS MESRS nécessite actuellement de soumettre une demande à un administrateur, qui provisionne manuellement la ressource et communique les identifiants par e-mail. Ce processus introduit des délais de plusieurs jours à plusieurs semaines, ne laisse aucune autonomie à l'utilisateur, ne génère aucune traçabilité et laisse la supervision entièrement à la charge de l'utilisateur une fois l'accès accordé.

Par ailleurs, les utilisateurs non experts — tels que les étudiants novices en cloud computing — n'ont aucun guide pour choisir la configuration de VM adaptée à leur charge de travail, ce qui crée un risque de sur-provisionnement (gaspillage de quota) ou de sous-provisionnement (performances dégradées).

1.3.2 Problématique

La problématique adressée par ce projet peut être formulée comme suit :

¿ Comment une plateforme web en libre-service peut-elle permettre aux utilisateurs académiques du cloud HCS MESRS de provisionner et gérer de façon autonome des machines virtuelles et des clusters Kubernetes, avec intégration du paiement, recommandation IA de configuration, suivi en temps réel du provisionnement et supervision automatisée — tout en opérant dans les contraintes de quota de ressources de l'environnement HCS partagé ? ¿

1.4 Étude de l'Existant

1.4.1 Présentation des Solutions Candidates

Plusieurs plateformes existantes abordent la problématique du libre-service cloud. Nous avons étudié quatre solutions représentatives pour identifier leurs capacités et leurs limites dans le contexte de l'environnement HCS MESRS.

OpenStack Horizon. Horizon est le tableau de bord web officiel d'OpenStack [5]. Il offre une interface de libre-service complète pour lancer des instances, gérer des réseaux et configurer des groupes de sécurité. HCS étant compatible OpenStack, Horizon pourrait théoriquement être déployé face aux API MESRS.

Proxmox VE. Proxmox Virtual Environment est une plateforme open source de gestion d'hyperviseur qui propose une interface web intégrée pour la gestion du cycle de vie des VM et conteneurs [6]. Elle est largement utilisée dans les établissements d'enseignement pour les environnements de laboratoire.

VMware Cloud Director. VMware Cloud Director propose un portail cloud multi-tenant pour l'infrastructure vSphere [7]. Il offre des capacités de libre-service étendues incluant les modèles de VM, la configuration réseau et la facturation.

AWS Management Console. La console de gestion AWS est la référence de facto des portails de libre-service cloud [8]. Elle propose une interface riche et intuitive pour tous les services AWS incluant EC2, EKS et IAM.

1.4.2 Analyse Comparative

Le tableau 1.1 résume les capacités clés de chaque solution face aux exigences du contexte MESRS.

TABLE 1.1. Comparaison des solutions cloud en libre-service existantes

Critère	OpenS-tack Horizon	Proxmox VE	VMware VCD	AWS Console	VM Mar- ketplace (pro- posé)
Compatibilité HCS	Partielle	✗	✗	✗	✓
Intégration paiement	✗	✗	✓	✓	✓
Recommandation IA	✗	✗	✗	✗	✓
Suivi SSE temps réel	✗	✗	✗	Partiel	✓
Terminal SSH navigateur	✗	✓	✗	✗	✓
Supervision automatisée	✗	Partiel	Partiel	Partiel	✓
Provisionnement K8s	Partiel	✗	Partiel	✓	✓
LLM local (sans API cloud)	✗	✗	✗	✗	✓
Open source / gratuit	✓	✓	✗	✗	✓

1.5 Critique de l'Existant

L'étude révèle les limites suivantes des plateformes existantes dans le contexte MESRS :

- **Absence d'intégration HCS native.** Aucune plateforme commerciale ne supporte le fournisseur Terraform huaweicloud/hcs ni les endpoints d'API spécifiques à HCS (type EIP External-01, résolution de flavor par nom).

- **Absence de guide IA.** Toutes les solutions existantes supposent que les utilisateurs connaissent déjà le type d'instance adapté à leur charge de travail. Les utilisateurs non experts ne bénéficient d'aucune assistance dans la plateforme.
- **Absence d'intégration du paiement.** À l'exception des offres commerciales (VMware VCD, AWS), aucune solution n'intègre de flux de paiement, pourtant nécessaires pour un modèle de crédit académique à l'usage.
- **Absence de suivi SSE en temps réel.** Les portails existants ne diffusent pas la progression étape par étape du provisionnement vers le navigateur, laissant les utilisateurs dans l'incertitude lors des opérations longues.
- **Absence d'intégration de supervision.** Les utilisateurs doivent installer leurs propres outils de monitoring après la création de la VM, ce qui requiert une expertise d'administration cloud que beaucoup d'utilisateurs académiques ne possèdent pas.
- **Confidentialité pour l'IA.** AWS, Azure et GCP proposent une assistance IA pour la sélection de ressources, mais ils envoient les descriptions de charge de travail des utilisateurs à des API cloud externes. Dans un contexte de recherche avec des données sensibles, un LLM hébergé localement est fortement préférable.

1.6 Solution Proposée

1.6.1 Vue d'Ensemble

VM Marketplace est une plateforme de gestion cloud en libre-service conçue spécifiquement pour l'environnement HCS MESRS. Elle permet aux utilisateurs authentifiés de configurer, payer, provisionner, accéder et superviser des machines virtuelles et des clusters Kubernetes entièrement via un navigateur, sans aucune intervention d'un administrateur.

La plateforme se différencie des solutions existantes sur quatre axes :

1. **Native HCS :** toutes les opérations de provisionnement utilisent le fournisseur Terraform officiel huaweicloud/hcs, garantissant une compatibilité totale avec l'environnement de production MESRS.
2. **Assistée par IA :** un LLM Llama 3.1 hébergé localement traduit des descriptions textuelles de charge de travail en recommandations concrètes de flavor HCS, rendant la plateforme accessible aux non-experts.
3. **Transparente :** des Server-Sent Events en temps réel diffusent chaque étape du provisionnement vers le navigateur de l'utilisateur, éliminant l'incertitude durant les 5 à 10 minutes d'exécution Terraform.
4. **Cycle de vie intégré :** du paiement à l'accès SSH en passant par les tableaux de bord de supervision et le téléchargement du kubeconfig, l'ensemble du cycle de vie est géré au sein d'une seule plateforme.

1.6.2 Architecture Générale

La plateforme se compose de trois services indépendants, comme illustré dans la figure 1.1.

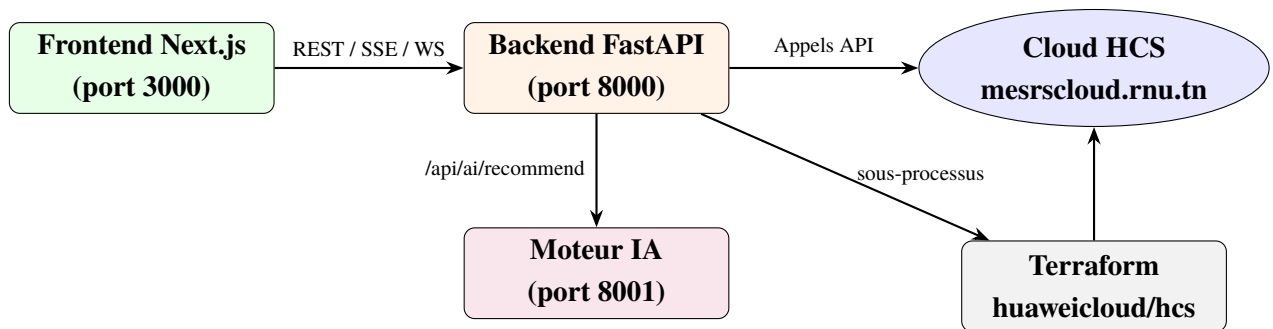


FIGURE 1.1. Architecture générale de VM Marketplace

1.7 Méthodologie

1.7.1 Comparaison des Méthodologies de Développement

Le tableau 1.2 compare trois méthodologies candidates face aux contraintes de ce projet : un développeur unique, une durée de six mois, des contraintes d'environnement HCS évolutives et une livraison de fonctionnalités incrémentale.

TABLE 1.2. Comparaison des méthodologies de développement

Critère	Cascade	Modèle en V	Agile/Scrum
Adaptabilité au changement	Faible	Faible	Élevée
Livraison de valeur précoce	Faible	Faible	Élevée
Adapté au développeur solo	Oui	Oui	Oui
Gestion des inconnues externes	Faible	Faible	Élevée
Charge documentaire	Élevée	Élevée	Modérée
Validation intégrée	En fin	Parallèle	À chaque sprint

1.7.2 Méthodologie Retenue : Agile Itératif

Compte tenu des nombreuses contraintes imprévues de l'environnement HCS (limitations de quota, politiques d'authentification SSH, règles de transfert réseau au niveau de l'hyperviseur), une approche de planification rigide aurait été inadaptée. Nous avons adopté une approche **Agile itérative** organisée en trois sprints, chacun livrant une tranche verticale fonctionnelle de la plateforme :

- **Sprint 1** : Authentification utilisateur, assistant de configuration de VM, provisionnement Terraform, intégration du paiement Stripe.
- **Sprint 2** : Moteur de recommandation IA, terminal SSH dans le navigateur, installation de la supervision Netdata.

- **Sprint 3** : Provisionnement de clusters Kubernetes avec solution réseau complète pour les nuds workers privés.

Chaque sprint a livré un ensemble de fonctionnalités démontrable et testable. Les problèmes découverts dans l’environnement HCS (épuisement du quota EIP, politique SSH à clé publique uniquement, `source_dest_check` au niveau de l’hyperviseur) ont été traités au sein du sprint par itération rapide.

1.7.3 Planification du Projet

La figure 1.2 présente le diagramme de Gantt du projet sur six mois.

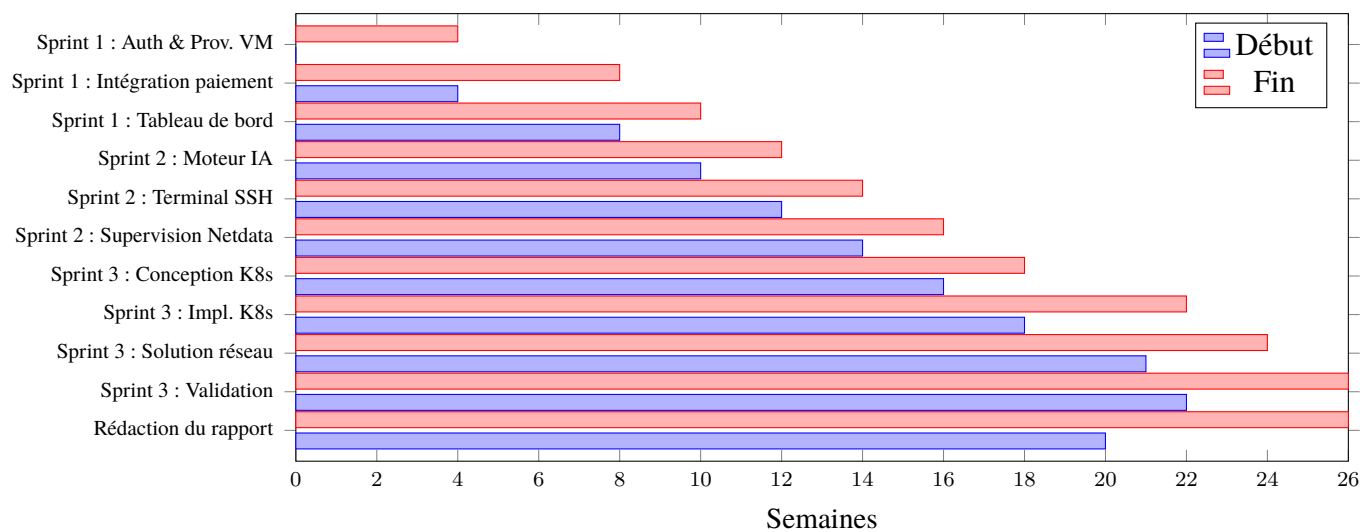


FIGURE 1.2. Diagramme de Gantt — planification du projet VM Marketplace ([TODO : ajuster les numéros de semaines selon la date de début réelle du stage])

1.8 Conclusion

Ce chapitre a situé VM Marketplace dans l’écosystème cloud académique du MESRS, identifié les lacunes des solutions existantes et défini le périmètre de la plateforme proposée. La méthodologie Agile itérative, organisée en trois sprints ciblés, a été choisie pour gérer les contraintes imprévisibles de l’environnement HCS de production. Le chapitre suivant formalise les besoins fonctionnels et non fonctionnels, et établit l’architecture technique de la solution.

Chapitre 2 Analyse et Architecture

2.1 Introduction

Ce chapitre formalise les besoins de VM Marketplace, identifie les acteurs du système, établit le backlog produit et décrit l'architecture technique ainsi que les choix technologiques qui sous-tendent l'implémentation.

2.2 Identification des Acteurs

La plateforme distingue deux acteurs :

Visiteur (utilisateur non authentifié) Toute personne accédant à l'application web VM Marketplace sans s'être authentifiée. Un visiteur peut accéder au moteur de recommandation IA et à la page marketplace pour explorer les modèles de VM, mais ne peut effectuer aucune opération de provisionnement ni de gestion.

Utilisateur authentifié (Chercheur / Étudiant) Toute personne ayant créé un compte et s'étant connectée. Un utilisateur authentifié peut provisionner des machines virtuelles et des clusters Kubernetes, gérer ses ressources via le tableau de bord, accéder aux terminaux SSH, consulter les tableaux de bord de supervision et télécharger les fichiers kubeconfig. Toutes les ressources provisionnées sont isolées par compte utilisateur.

2.3 Besoins Fonctionnels

2.3.1 Module Authentification

- En tant que visiteur, je veux m'inscrire avec un e-mail et un mot de passe afin d'accéder aux fonctionnalités de provisionnement de la plateforme.
- En tant que visiteur, je veux me connecter avec mes identifiants afin de recevoir un jeton JWT valide pendant 30 minutes.
- En tant qu'utilisateur authentifié, je veux que ma session soit sécurisée par un jeton Bearer JWT afin que mes ressources restent privées.

2.3.2 Module Provisionnement de VM

- En tant qu'utilisateur authentifié, je veux sélectionner une configuration de VM via un assistant en plusieurs étapes (type d'instance, OS, stockage, réseau, région, révision) afin de personnaliser ma machine avant de valider le paiement.

- En tant qu'utilisateur authentifié, je veux payer ma VM via un formulaire Stripe afin que le coût soit débité de ma carte avant le début du provisionnement.
- En tant qu'utilisateur authentifié, je veux voir une progression en temps réel étape par étape (via Server-Sent Events) pendant le provisionnement afin de connaître l'état actuel sans rafraîchir la page.
- En tant qu'utilisateur authentifié, je veux recevoir un e-mail de confirmation une fois ma VM provisionnée afin de disposer des informations d'accès dans un format durable.

2.3.3 Module Gestion des VM

- En tant qu'utilisateur authentifié, je veux consulter toutes mes VM provisionnées dans un tableau de bord afin de les gérer depuis un seul endroit.
- En tant qu'utilisateur authentifié, je veux supprimer une VM depuis le tableau de bord afin que les ressources HCS correspondantes soient détruites via `terraform destroy`.
- En tant qu'utilisateur authentifié, je veux ouvrir un terminal SSH dans le navigateur vers ma VM afin d'effectuer des opérations d'administration sans installer de client SSH.
- En tant qu'utilisateur authentifié, je veux consulter le tableau de bord de supervision Netdata de ma VM (CPU, mémoire, disque, réseau) intégré dans la plateforme afin d'évaluer l'utilisation des ressources en un coup d'il.

2.3.4 Module Recommandation IA

- En tant que visiteur ou utilisateur authentifié, je veux décrire ma charge de travail en langage naturel afin que le moteur IA recommande le flavor et la configuration de VM appropriés.
- En tant que visiteur ou utilisateur authentifié, je veux voir le niveau de confiance et le raisonnement de l'IA avec la recommandation afin d'évaluer si elle correspond à mes besoins.

2.3.5 Module Cluster Kubernetes

- En tant qu'utilisateur authentifié, je veux configurer un cluster Kubernetes (flavor maître, flavor worker, nombre de workers, taille disque, région) via un assistant afin de déployer des clusters multi-nuds sans expertise Kubernetes.
- En tant qu'utilisateur authentifié, je veux payer le cluster via Stripe puis suivre la progression du provisionnement étape par étape afin de savoir quand le cluster est prêt.
- En tant qu'utilisateur authentifié, je veux télécharger le fichier kubeconfig de mon cluster depuis le tableau de bord afin de me connecter avec `kubectl` depuis ma machine locale.
- En tant qu'utilisateur authentifié, je veux supprimer un cluster depuis le tableau de bord afin que toutes les ressources HCS associées soient détruites proprement.

2.4 Besoins Non Fonctionnels

TABLE 2.1. Besoins non fonctionnels

Catégorie	Besoin	Cible
Sécurité	Authentification JWT	Jetons expirant après 30 min ; hachage bcrypt des mots de passe
Sécurité	Paiement	Données de carte gérées exclusivement par l'API PCI Stripe
Sécurité	Isolation SSH	Chaque cluster utilise une paire de clés RSA unique générée par session
Performance	Temps de réponse API	Tous les endpoints hors provisionnement répondent en moins de 500 ms
Performance	Recommandation IA	Recommandation retournée en moins de 60 secondes
Ergonomie	UX de l'assistant	Formulaire multi-étapes avec validation à chaque étape ; aucune donnée perdue en navigation arrière
Fiabilité	Suivi de progression	Le flux SSE maintient la connexion pendant toute la durée du provisionnement (5–10 min)
Scalabilité	Utilisateurs concurrents	Chaque provisionnement s'exécute dans un thread en arrière-plan ; l'API reste réactive
Maintenabilité	Isolation IaC	Chaque VM/cluster possède son propre répertoire terraform_states/

2.5 Diagramme de Cas d'Utilisation Global

La figure 2.1 présente le diagramme de cas d'utilisation global de VM Marketplace.

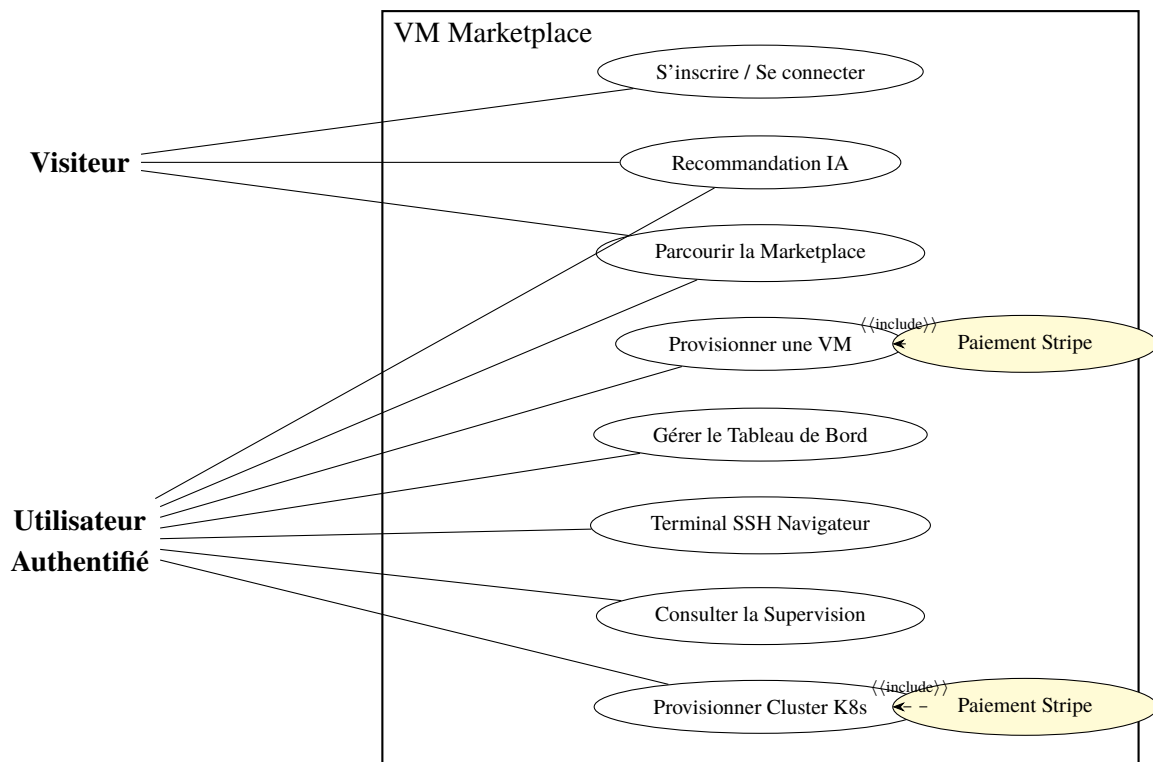


FIGURE 2.1. Diagramme de cas d'utilisation global — VM Marketplace

2.6 Diagramme de Classes

Le modèle de données central est composé de trois entités ORM SQLAlchemy stockées dans une base de données SQLite, comme illustré à la figure 2.2.

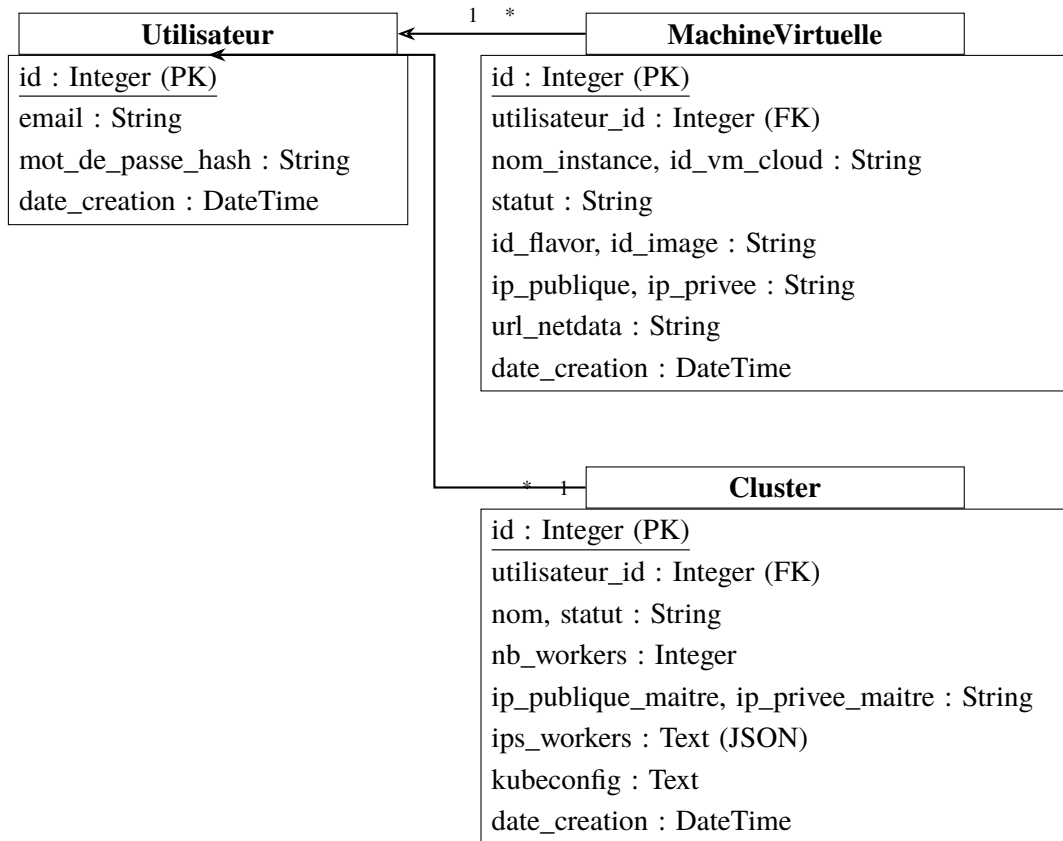


FIGURE 2.2. Modèle de données principal (entités ORM SQLAlchemy)

2.7 Backlog Produit

Le tableau 2.2 présente le backlog produit organisé par sprint.

TABLE 2.2. Backlog produit

ID	Sprint	Histoire Utilisateur	Priorité
US-01	1	S’inscrire et se connecter avec e-mail / mot de passe	Critique
US-02	1	Configurer une VM via l’assistant multi-étapes	Critique
US-03	1	Payer via le formulaire Stripe	Critique
US-04	1	Provisionner la VM sur HCS via Terraform	Critique
US-05	1	Diffuser la progression du provisionnement via SSE	Haute
US-06	1	Recevoir un e-mail de confirmation après provisionnement	Moyenne
US-07	1	Consulter et supprimer ses VM dans le tableau de bord	Critique
US-08	2	Décrire sa charge de travail et recevoir une reco IA	Haute
US-09	2	Parcourir les modèles de VM préconfigurés (marketplace)	Moyenne
US-10	2	Ouvrir un terminal SSH navigateur vers sa VM	Haute
US-11	2	Consulter le tableau de bord Netdata par VM	Haute
US-12	3	Configurer et payer un cluster k3s	Critique
US-13	3	Provisionner le cluster k3s sur HCS via Terraform + SSH	Critique
US-14	3	Diffuser la progression du provisionnement k3s via SSE	Haute
US-15	3	Télécharger le kubeconfig depuis le tableau de bord	Critique
US-16	3	Supprimer un cluster k3s depuis le tableau de bord	Haute
US-17	3	Ouvrir un terminal SSH navigateur vers les nuds du cluster	Haute
US-18	3	Activer l’autoguérison du cluster (contrôleur hub-spoke)	Moyenne
US-19	3	Synchroniser la BD avec l’état Terraform réel du cloud	Moyenne
US-20	3	Importer dans la BD les ressources cloud non enregistrées	Moyenne

2.8 Architecture du Système

2.8.1 Architecture Logique

VM Marketplace adopte une **architecture client-serveur à trois services** avec une séparation claire des responsabilités :

1. **Frontend Next.js (port 3000)** — rend l’interface utilisateur, gère l’état global via React Context, communique avec le backend via REST et SSE, et affiche un terminal WebSocket xterm.js.
2. **Backend FastAPI (port 8000)** — gère l’authentification, le CRUD des VM et clusters, le traitement des paiements, l’orchestration des sous-processus Terraform et l’installation Netdata via SSH.
3. **Moteur IA (port 8001)** — processus FastAPI distinct qui envoie des prompts en langage naturel à une instance Ollama locale, analyse la réponse JSON du LLM et la mappe vers une recommandation de flavor HCS.

Le moteur IA est intentionnellement isolé du backend principal afin d’éviter de bloquer l’API durant l’inférence LLM, qui peut prendre 10 à 30 secondes.

2.8.2 Architecture de Déploiement

La figure 2.3 présente l'architecture de déploiement avec toutes les dépendances externes.

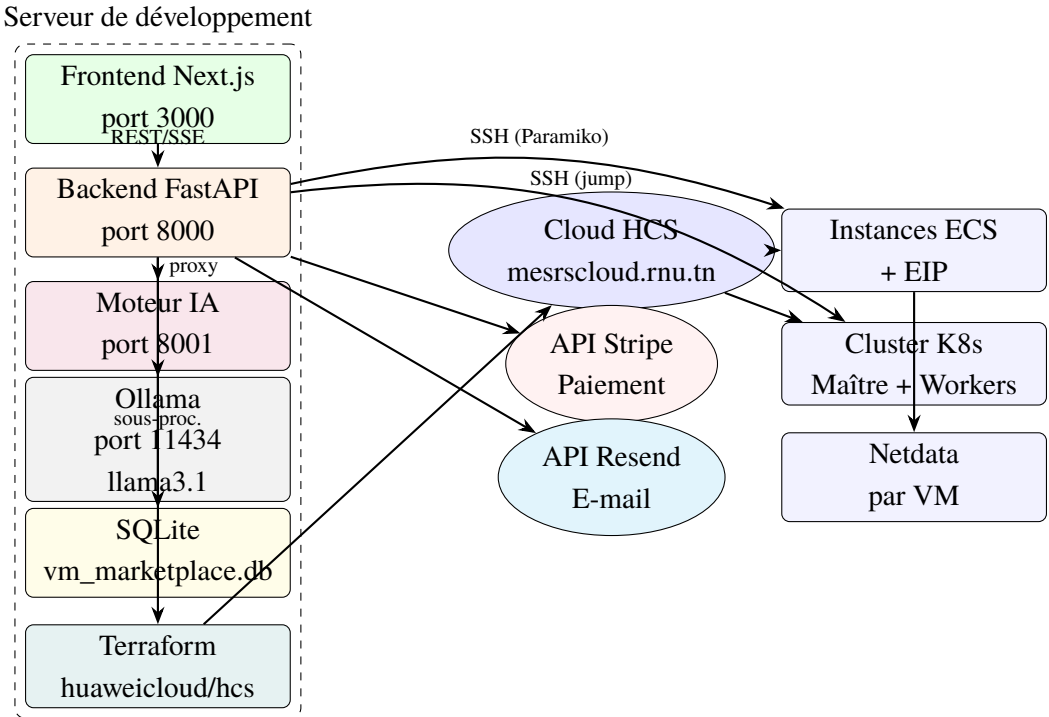


FIGURE 2.3. Architecture de déploiement de VM Marketplace

2.9 Environnement de Travail

TABLE 2.3. Environnement de développement

Catégorie	Outils / Versions
Machine de développement	Windows 11 / macOS (projet multiplateforme)
IDE	Visual Studio Code avec extensions Python, ESLint, Prettier
Contrôle de version	Git + GitHub (dépôt <code>vm_marketplace</code>)
Plateforme cloud	Huawei Cloud Stack — <code>mesrscloud.rnu.tn</code> , région <code>tn-global-1</code>
Terraform	v1.x avec fournisseur <code>huaweicloud/hcs</code> v2.4.0
Python	3.11
Node.js	18 LTS
Base de données	SQLite 3 (développement); fichier <code>vm_marketplace.db</code>
Runtime LLM	Ollama v0.x avec le modèle <code>llama3.1</code>

2.10 Choix Technologiques

2.10.1 Frontend : Next.js 14

Next.js a été choisi pour son routage basé sur les fichiers, ses capacités de rendu côté serveur et son écosystème mature. TypeScript garantit la sécurité des types à toutes les frontières d'API. Tailwind CSS offre un style utilitaire rapide. Stripe.js et la bibliothèque Stripe Elements gèrent la collecte des données de carte conforme PCI dans le navigateur [9], [10].

2.10.2 Backend : FastAPI

FastAPI a été retenu pour sa gestion asynchrone des requêtes, sa documentation OpenAPI automatique et ses annotations de types Python natives avec validation Pydantic. Comparé à Flask (synchrone, moins structuré) et Django (couplage ORM plus fort), FastAPI offre le meilleur équilibre entre performance et productivité pour un service orienté API [11]. SQLAlchemy fournit la couche ORM pour SQLite.

2.10.3 Infrastructure as Code : Terraform

Terraform a été choisi pour le provisionnement des VM et des clusters car il fournit des descriptions d'infrastructure déclaratives, une gestion d'état et des opérations apply idempotentes. Le fournisseur huaweicloud/hcs (v2.4.0) supporte tous les types de ressources HCS requis [4].

2.10.4 Moteur IA : Ollama + Llama 3.1

Le moteur de recommandation IA utilise Ollama pour servir le modèle Llama 3.1 (8B) localement [12], [13]. Cela évite d'envoyer des descriptions de charge de travail potentiellement sensibles à des API LLM cloud externes. Le moteur analyse la sortie JSON structurée du LLM et la mappe vers un flavor HCS concret via un module mapper.py personnalisé.

2.10.5 Connectivité SSH : Paramiko

Paramiko [14] est utilisé à deux fins : (1) l'installation post-provisionnement de Netdata sur les VM individuelles via SSH direct ; (2) la connexion SSH aux nuds workers en IP privée via le maître comme hôte de rebond, en utilisant `transport.open_channel("direct-tcpip", ...)`.

2.10.6 Supervision : Netdata

Netdata [15] a été retenu pour son installation agent sans configuration et son tableau de bord web intégré accessible sur le port 19999. Après installation, l'URL Netdata de la VM est

stockée en base de données et affichée sous forme d'iframe dans la page de supervision de la plateforme.

2.11 Conclusion

Ce chapitre a formalisé les besoins, le modèle de données, le backlog produit et l'architecture technique de VM Marketplace. L'architecture à trois services sépare clairement le rendu de l'interface, la logique métier et l'inférence IA, tandis que Terraform délègue toute la gestion des ressources HCS à un outil IaC déclaratif. Le chapitre suivant présente les décisions de conception et d'implémentation prises lors de chacun des trois sprints de développement.

Chapitre 3 Conception et Réalisation

3.1 Introduction

Ce chapitre documente les trois sprints de développement à travers lesquels VM Marketplace a été construite. Chaque section de sprint s'ouvre sur son objectif et son backlog, détaille les décisions de conception et les artefacts d'implémentation clés, et se clôt par les difficultés rencontrées et la façon dont elles ont été résolues.

3.2 Sprint 1 — Authentification et Provisionnement de VM

3.2.1 Objectif du Sprint

Livrer un flux complet de bout en bout, de l'inscription de l'utilisateur à une VM provisionnée et opérationnelle sur HCS, incluant le paiement Stripe et la diffusion SSE de progression en temps réel.

3.2.2 Backlog du Sprint

TABLE 3.1. Backlog Sprint 1

ID	Statut	Description
US-01	Terminé	S'inscrire et se connecter avec e-mail / mot de passe
US-02	Terminé	Configurer une VM via l'assistant multi-étapes
US-03	Terminé	Payer via le formulaire Stripe
US-04	Terminé	Provisionner la VM sur HCS via Terraform
US-05	Terminé	Diffuser la progression du provisionnement via SSE
US-06	Terminé	Envoyer un e-mail de confirmation après provisionnement
US-07	Terminé	Consulter et supprimer ses VM dans le tableau de bord

3.2.3 Conception de l'Authentification

Hachage du mot de passe. Les mots de passe des utilisateurs sont hachés avec bcrypt (10 rounds) via la bibliothèque passlib. Le hash est stocké dans la table utilisateurs; le mot de passe en clair n'est jamais persisté.

Jetons JWT. À la connexion réussie, le backend génère un JWT signé contenant l'identifiant (sub) et la date d'expiration (exp = heure actuelle + 30 minutes), en utilisant l'algorithme

HS256. Le frontend stocke le jeton dans le localStorage et l'attache en tant qu'en-tête Bearer à toutes les requêtes API ultérieures.

La figure 3.1 présente le diagramme de séquence d'authentification.

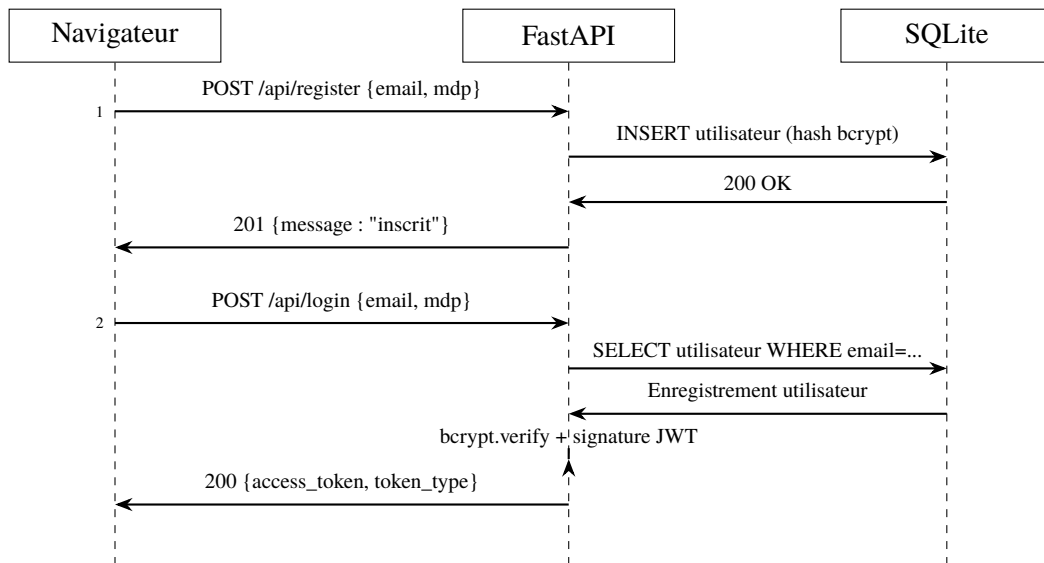


FIGURE 3.1. Diagramme de séquence — Authentification

3.2.4 Assistant de Configuration de VM

L'assistant est implémenté comme un formulaire React multi-étapes en sept étapes : sélection du type d'instance, sélection du système d'exploitation, configuration du stockage, configuration réseau, sélection de la région, révision et paiement. L'état est géré globalement dans un contexte React `BuildVMContext` (`frontend/BuildVMContext.tsx`), garantissant qu'aucune donnée n'est perdue lors de la navigation entre les étapes.

[Screenshot : Assistant de configuration de VM en plusieurs étapes — Étape 1 : Sélection du type d'instance]

Les images de VM sont répertoriées avec leurs UUID HCS directement dans le frontend. Le catalogue distingue les images disponibles de celles marquées `needs_id` (UUID non encore attribué dans l'environnement HCS de production).

3.2.5 Pipeline de Provisionnement de VM

La figure 3.2 illustre la séquence de provisionnement de VM, du paiement confirmé à la VM en cours d'exécution.

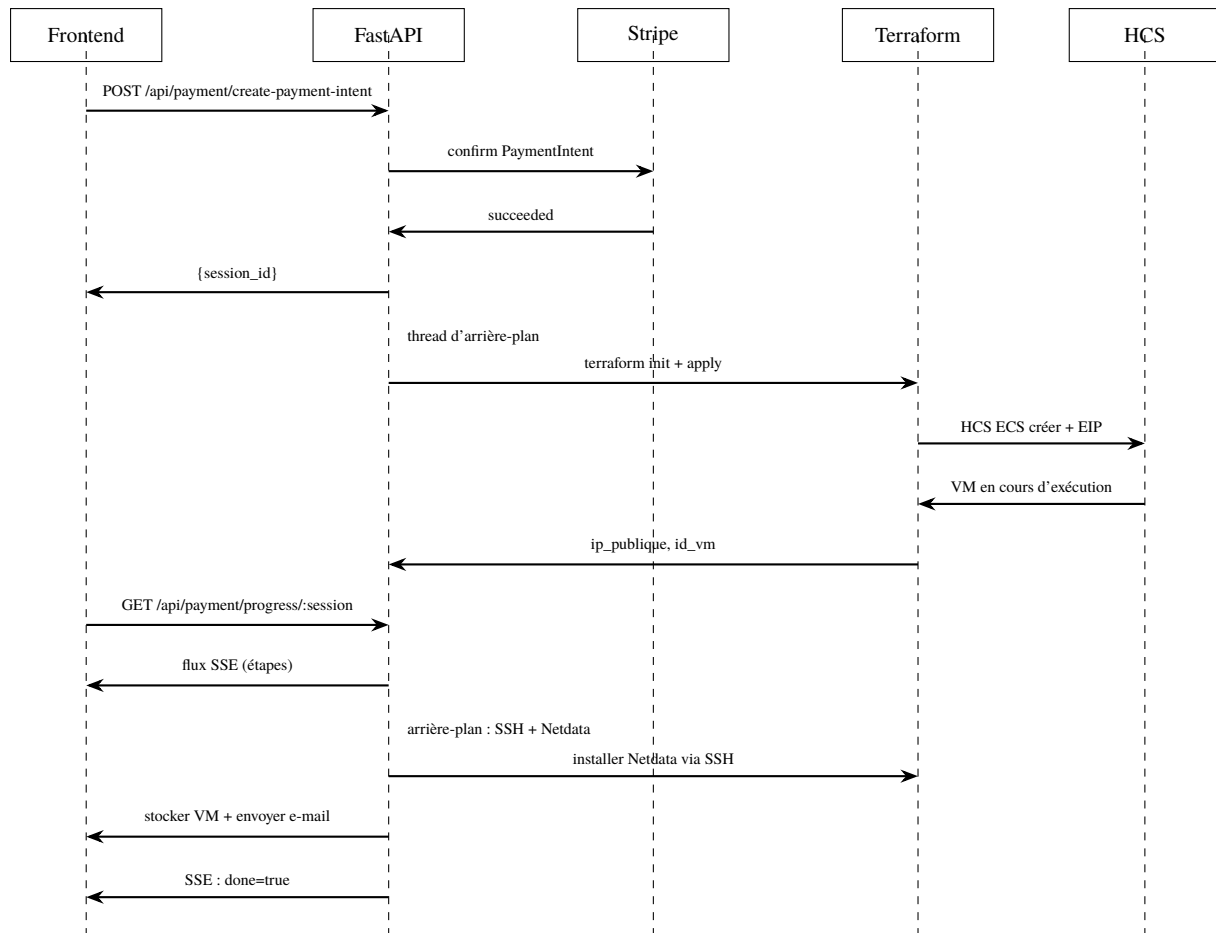


FIGURE 3.2. Diagramme de séquence — Provisionnement de VM

Modèle Terraform. Chaque VM possède son propre répertoire d'état Terraform à `terraform_states/<id_vm>`. Le modèle (`terraform_test/main.tf`) déclare une `hcs_ecs_compute_instance` avec un disque système, un EIP (`hcs_vpc_eip`) et une association EIP. Le flavor est résolu par nom ou par ID via une requête de source de données sur tous les flavors HCS disponibles.

Authentification SSH root via cloud-init. Les images Ubuntu 22.04 sur HCS imposent `PasswordAuthentication` no par défaut. Le champ `user_data` passe un script bash à `cloud-init` au premier démarrage pour réactiver le SSH root par mot de passe, permettant au backend de se connecter pour l'installation de Netdata.

Suivi de progression. La progression du provisionnement est suivie dans un dictionnaire Python en mémoire `progress_store` dans `backend/app/api/payment.py`. Chaque étape (paiement confirmé, infrastructure provisionnée, supervision installée) est mise à jour par le thread d'arrière-plan. Le frontend suit le flux SSE `GET /api/payment/progress/:session_id` et affiche un indicateur d'étapes.

[Screenshot : Indicateur de progression SSE en temps réel lors du provisionnement de VM]

3.2.6 Tableau de Bord

Le tableau de bord (`frontend/app/dashboard/`) liste toutes les VM appartenant à l'utilisateur authentifié. Chaque carte VM affiche le statut, l'IP publique, le flavor, la date de création et des boutons d'action pour le SSH, la supervision et la suppression.

[Screenshot : Tableau de bord utilisateur — liste des VM avec boutons d'action]

3.2.7 Difficultés Rencontrées dans le Sprint 1

- **Nom vs. ID de flavor** : L'API HCS exige des ID de flavor (UUID), mais les utilisateurs et les modèles de l'assistant utilisent des noms de flavor (ex. : `c7.large.4`). Résolu en interrogeant `data.hcs_ecs_compute_flavors.all` et en faisant correspondre par nom ou ID au moment du plan Terraform.
- **Type d'EIP incorrect** : Le type d'IP publique `5_bgp` utilisé dans certains exemples Terraform n'est pas valide sur le déploiement HCS MESRS ; le type correct est `External-01`. Résolu en lisant les valeurs de l'interface portail HCS.

3.3 Sprint 2 — Recommandation IA, Terminal SSH et Supervision

3.3.1 Objectif du Sprint

Ajouter le moteur de recommandation IA, un terminal SSH dans le navigateur, la supervision automatisée Netdata et une marketplace de modèles de VM préconfigurés.

3.3.2 Backlog du Sprint

TABLE 3.2. Backlog Sprint 2

ID	Statut	Description
US-08	Terminé	Décrire sa charge de travail, recevoir une recommandation IA de VM
US-09	Terminé	Parcourir les modèles de VM préconfigurés
US-10	Terminé	Terminal SSH dans le navigateur
US-11	Terminé	Tableau de bord de supervision Netdata par VM

3.3.3 Moteur de Recommandation IA

Le moteur IA s'exécute en tant que processus FastAPI distinct sur le port 8001 afin d'isoler la latence d'inférence LLM de l'API principale. Il est composé de trois modules :

1. `ollama_client.py` — envoie la description textuelle de la charge de travail de l'utilisateur à l'API HTTP Ollama locale (`http://localhost:11434`) avec un prompt structuré demandant à Llama 3.1 de retourner un objet JSON.
2. `parser.py` — extrait et valide les champs JSON de la réponse du LLM : `workload` (intensité CPU/mémoire/GPU), `users` (nombre d'utilisateurs concurrents), `app_type`, `budget` et `confidence` (flottant entre 0 et 1).
3. `mapper.py` — mappe les champs analysés vers un flavor HCS et une spécification de configuration. L'intensité de la charge de travail (`low/medium/high`) sélectionne un niveau de flavor (ex. : `c7.large.4` pour les charges de calcul moyennes).

La figure 3.3 présente le flux de données de la recommandation IA.

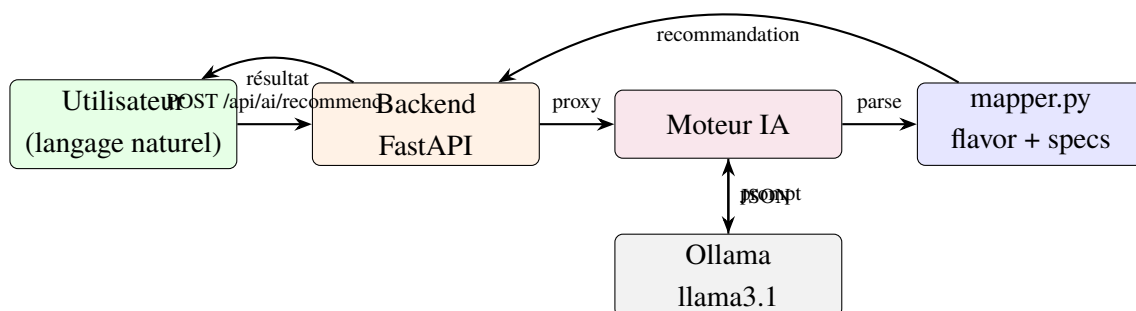


FIGURE 3.3. Flux de données de la recommandation IA

[Screenshot : Page de recommandation IA — l'utilisateur saisit sa description de charge de travail, le système retourne la spécification de VM recommandée avec le score de confiance]

3.3.4 Terminal SSH dans le Navigateur

La fonctionnalité de terminal SSH offre un accès shell root à toute VM provisionnée directement dans le navigateur, sans nécessiter l'installation d'un client SSH ni la gestion de clés privées.

L'implémentation repose sur deux composants :

- **Frontend** : La bibliothèque `xterm.js` rend un émulateur de terminal dans le navigateur et se connecte au backend via une connexion WebSocket.
- **Backend** (`api/ssh.py`) : Un endpoint WebSocket accepte la connexion, ouvre une session SSH Paramiko vers la VM cible (en utilisant l'IP publique de la VM et les identifiants root stockés), démarre un canal shell interactif et transfère les octets de manière bidirectionnelle entre le WebSocket et le canal SSH.

[Screenshot : Terminal SSH dans le navigateur affichant un shell root sur une VM HCS provisionnée]

3.3.5 Supervision Netdata

Après le provisionnement de la VM, un second thread d'arrière-plan se connecte à la VM via SSH et installe l'agent Netdata :

Listing 3.1. Commande d'installation de Netdata (exécutée via SSH)

```
1 bash <(curl -Ss https://my-netdata.io/kickstart.sh) --non-interactive
```

Le tableau de bord Netdata (port 19999) est ensuite accessible à `http://<ip_public>:19999`. L'URL est stockée dans la colonne `netdata_url` de la table `virtual_machines` et affichée sous forme d'iframe sandboxé dans la page de supervision de la plateforme.

[Screenshot : Tableau de bord de supervision Netdata intégré dans la plateforme pour une VM en cours d'exécution]

3.3.6 Marketplace de VM

La page marketplace (`frontend/app/marketplace/`) affiche un catalogue de modèles de VM préconfigurés (ex. : `Station Data Science`, `Serveur Web`, `Nud ML Haute Mémoire`) avec les flavors recommandés, le système d'exploitation, la taille du disque et le coût estimé. La sélection d'un modèle pré-remplit l'assistant de création de VM.

3.4 Sprint 3 — Clusters k3s, Autoguérison et Synchronisation Cloud

3.4.1 Objectif du Sprint

Concevoir et implémenter un pipeline complet de provisionnement de clusters k3s multi-nuds sur HCS, intégrer un terminal SSH dédié aux clusters, proposer un contrôleur d'autoguérison en option et ajouter des mécanismes de synchronisation entre la base de données et l'état Terraform réel dans le cloud.

3.4.2 Backlog du Sprint

TABLE 3.3. Backlog Sprint 3

ID	Statut	Description
US-12	Terminé	Configurer et payer un cluster k3s
US-13	Terminé	Provisionner le cluster k3s sur HCS via Terraform + SSH
US-14	Terminé	Diffuser la progression du provisionnement k3s via SSE
US-15	Terminé	Télécharger le kubeconfig depuis le tableau de bord
US-16	Terminé	Supprimer un cluster k3s depuis le tableau de bord
US-17	Terminé	Terminal SSH navigateur vers les nuds du cluster
US-18	Terminé	Activer l'autoguérison du cluster (contrôleur hub-spoke)
US-19	Terminé	Synchroniser la BD avec l'état Terraform réel du cloud
US-20	Terminé	Importer dans la BD les ressources cloud non enregistrées

3.4.3 Choix de k3s comme Distribution Kubernetes

k3s [16] est la distribution Kubernetes légère de Rancher Labs. Comparé à kubeadm, k3s offre plusieurs avantages décisifs dans le contexte HCS MESRS :

- **Installation en une commande** : un seul script `curl | sh` installe et démarre le serveur k3s (maître) ou l'agent (worker), sans étapes de pré-configuration des modules noyau ni gestion de `containerd`.
- **Réseau intégré** : k3s embarque Flannel CNI, supprimant l'étape `kubect1 apply -f flannel.yaml` séparée.
- **Binaire unique** : k3s est distribué comme un binaire statique, éliminant les dépendances de paquets apt et les problèmes de dépôt miroir fréquents dans les environnements réseau contraints.
- **Empreinte réduite** : convient aux flavors académiques de taille modeste (4 vCPU, 16 Go RAM).

3.4.4 Architecture du Cluster

Un cluster k3s sur HCS est composé d'un nud maître (avec une EIP) et de N nuds workers (IP privée uniquement). Cette topologie asymétrique a été imposée par le quota EIP du projet HCS partagé MESRS : une seule EIP par cluster pouvait être allouée.

Le nud maître remplit simultanément trois rôles :

1. **Serveur API Kubernetes** (port 6443) — accessible depuis l'extérieur via l'EIP du maître.
2. **Hôte de rebond (bastion)** — le backend se connecte aux nuds workers via la session SSH du maître en utilisant l'API Paramiko `transport.open_channel("direct-tcpip", ...)`.
3. **Passerelle NAT** — le maître transfère les paquets à destination d'Internet provenant des nuds workers, grâce à des règles iptables `MASQUERADE`.

La figure 3.4 illustre l'architecture réseau du cluster.

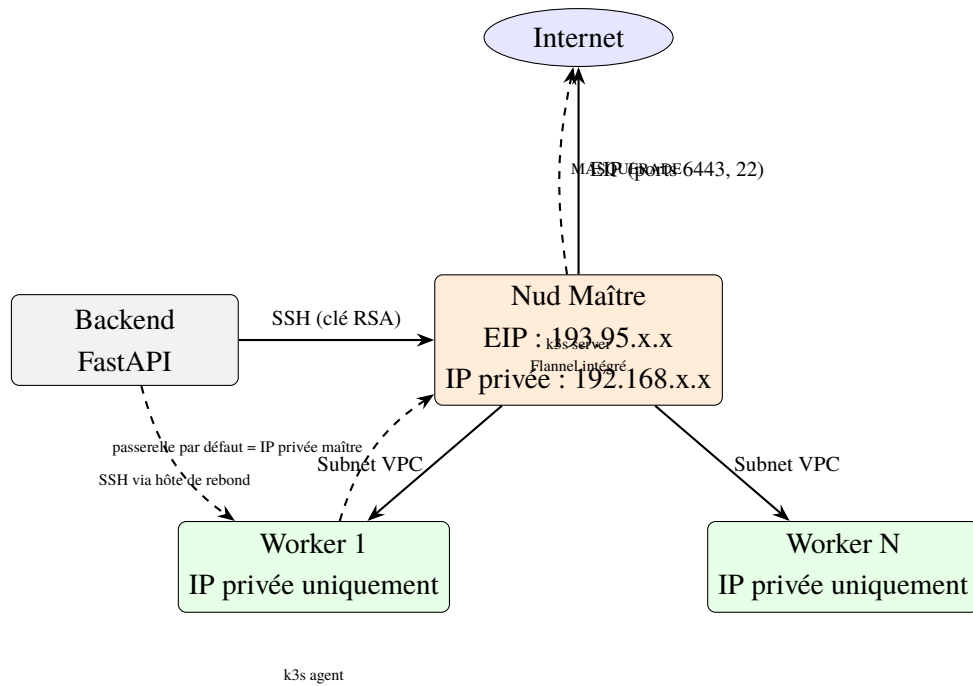


FIGURE 3.4. Architecture réseau du cluster k3s sur HCS

3.4.5 Modèle Terraform du Cluster

Le modèle Terraform du cluster (terraform_cluster/main.tf) provisionne :

- Une instance maître `hcs_ecs_compute_instance` avec `source_dest_check = false` (requis pour le transfert de paquets NAT).
- Un EIP maître et une association EIP.
- N instances workers `hcs_ecs_compute_instance` (sans EIP).
- Tous les nuds reçoivent une clé publique RSA générée dynamiquement via cloud-config dans `user_data`, utilisée pour le SSH sans mot de passe.

Génération dynamique de paire de clés SSH. Une paire de clés RSA 2048 bits fraîche est générée par session de provisionnement via Paramiko :

Listing 3.2. Génération de paire de clés RSA par cluster

```

1 def _generate_ssh_keypair():
2     key = paramiko.RSAKey.generate(2048)
3     public_key_line = f"ssh-rsa {key.get_base64()} k3s-cluster"
4     return key, public_key_line

```

La clé publique est injectée dans les nuds via cloud-config :

Listing 3.3. user_data cloud-init pour l'injection de clé SSH

```

1 #cloud-config
2 disable_root: false

```

```

3  ssh_authorized_keys:
4    - ssh-rsa AAAA... k3s-cluster

```

3.4.6 Séquence d'Initialisation k3s

La fonction `install_k3s()` dans `backend/app/services/cluster_service.py` exécute les étapes suivantes via SSH :

1. **Attente de 120 s** pour que les VM terminent `cloud-init` et deviennent joignables.
2. **Connexion au maître** via SSH direct en utilisant la clé RSA provisionnée.
3. **Activation du NAT sur le maître** : détection de l'interface réseau par défaut, activation du `ip_forward` et ajout d'une règle `MASQUERADE` `iptables`.
4. **Installation de k3s server sur le maître** : une seule commande installe et démarre le serveur k3s avec Flannel CNI intégré.

Listing 3.4. Installation k3s server sur le nud maître

```

1  curl -sfl https://get.k3s.io | INSTALL_K3S_EXEC=\
2    "server --tls-san <EIP_MAÎTRE> --node-ip <IP_PRIVÉE>"
    sh -

```

5. **Récupération du token de jonction** depuis `/var/lib/rancher/k3s/server/node-token` sur le maître.
6. **Connexion aux workers via hôte de rebond** (canal `direct-tcpip` Paramiko à travers le maître).
7. **Configuration des workers** : remplacement de la passerelle par défaut par l'IP privée du maître, remplacement du DNS (`/etc/resolv.conf`) par les DNS Google (8.8.8.8).
8. **Installation de k3s agent sur chaque worker** :

Listing 3.5. Installation k3s agent sur un nud worker

```

1  curl -sfl https://get.k3s.io | K3S_URL=https://<
    IP_PRIVÉE_MAÎTRE>:6443 \
2    K3S_TOKEN=<TOKEN> sh -

```

9. **Récupération du kubeconfig** depuis `/etc/rancher/k3s/k3s.yaml` et réécriture de l'URL du serveur API de l'IP privée vers l'EIP du maître.

3.4.7 Solution Réseau : Maître comme Passerelle NAT

Problème. Les nuds workers n'ont pas d'EIP et donc aucune route internet directe. Sans accès à Internet, `apt-get` ne peut pas télécharger les paquets K8s, et `containerd` ne peut pas récupérer les images de conteneurs.

Investigation. Les tentatives initiales d’activation du transfert NAT avec uniquement des règles MASQUERADE iptables sur le maître se sont révélées insuffisantes. Les paquets envoyés par les workers via le maître étaient silencieusement abandonnés malgré la règle iptables active.

Cause racine. L’hyperviseur HCS applique un contrôle `source_dest_check` sur chaque carte réseau de VM : les paquets arrivant sur la carte du maître dont l’IP de destination n’est pas celle du maître lui-même sont rejetés au niveau de l’hyperviseur, avant d’atteindre la pile réseau Linux. Il s’agit d’un mécanisme anti-usurpation qui ne peut pas être désactivé depuis l’intérieur de la VM.

Solution. L’attribut `source_dest_check = false` a été ajouté au bloc `network` du maître dans Terraform, ce qui désactive le contrôle au niveau de l’hyperviseur via l’API HCS :

Listing 3.6. Terraform : désactivation de `source_dest_check` sur la carte réseau du maître

```
1 resource "hcs_ecs_compute_instance" "master" {
2   ...
3   network {
4     uuid          = var.subnet_id
5     source_dest_check = false # autorise le transfert NAT
6   }
7 }
```

Avec cette modification, le trafic internet des workers est correctement acheminé via la règle MASQUERADE du maître vers Internet.

3.4.8 Assistant et Tableau de Bord du Cluster

L’assistant de configuration du cluster (`frontend/app/build-cluster/`) collecte le nom du cluster, les flavors du maître et des workers, le nombre de workers, l’image OS, la taille du disque et le mot de passe administrateur (avec confirmation). Après le paiement via Stripe, la progression du provisionnement est diffusée vers le frontend via un endpoint SSE dédié (`GET /api/clusters/progress/:session_id`).

[Screenshot : Assistant de provisionnement du cluster k3s — configuration des nuds maître et workers]

[Screenshot : Flux de progression SSE du cluster k3s — les étapes sont complètes]

Une fois le cluster en statut running, le tableau de bord l’affiche aux côtés des VM ordinaires avec un bouton Télécharger le kubeconfig.

[Screenshot : Tableau de bord affichant un cluster k3s en cours d’exécution avec l’IP du maître et le nombre de workers]

3.4.9 Terminal SSH pour Clusters

La page de gestion de clés SSH des clusters (`frontend/app/ssh/cluster/`) permet à l’utilisateur d’accéder à un terminal interactif vers le nud maître ou tout nud worker de son cluster, directement depuis le navigateur.

L’implémentation réutilise la même infrastructure WebSocket-xterm.js développée pour les VM individuelles au Sprint 2. La connexion au maître s’effectue via SSH direct ; la connexion aux workers transite par le maître comme hôte de rebond (`transport.open_channel("direct-tcpip", ...)` Paramiko), en utilisant la clé RSA privée stockée en base de données lors du provisionnement.

[Screenshot : Terminal SSH dans le navigateur — session interactive sur le nud maître d’un cluster k3s]

3.4.10 Contrôleur d’Autoguérison (Self-Healing)

La plateforme propose en option payante (\$15/mois) un contrôleur d’autoguérison basé sur un modèle hub-spoke. Lorsque l’utilisateur active cette option dans l’assistant de cluster, un processus d’enregistrement est déclenché à la fin du provisionnement.

Architecture hub-spoke.

- **Hub** : un contrôleur centralisé déjà déployé en production surveille en permanence les clusters qui lui sont enregistrés.
- **Spoke** : le cluster k3s nouvellement provisionné est enregistré auprès du hub via la fonction `register_spoke_with_hub()` du service `backend/app/services/self_healing_service.py`.

Processus d'enregistrement. Après l'initialisation réussie de k3s, le service d'autoguérison :

1. Applique le manifest RBAC `spoke-rbac.yaml` sur le cluster cible (création d'un `ClusterRole` et d'un `ClusterRoleBinding` accordant au hub les droits de lecture sur les ressources de charge de travail).
2. Envoie les coordonnées du cluster (IP maître, kubeconfig, credentials) au hub via un appel HTTP authentifié.
3. Met à jour la colonne `self_healing_status` du cluster en base de données (pending, registered ou error).

Le statut d'enregistrement est visible dans le tableau de bord et dans l'assistant cluster. En cas d'erreur d'enregistrement, le message est exposé dans le champ `self_healing_error` et affiché à l'utilisateur.

[Screenshot : Option autoguérison dans l'assistant de cluster — carte d'activation avec badge Recommandé et récapitulatif de prix]

3.4.11 Synchronisation et Import depuis le Cloud

Deux problèmes de cohérence entre la base de données et l'état Terraform réel ont été identifiés lors de la validation :

- Des VM ou clusters supprimés directement depuis la console HCS (hors plateforme) continuent d'apparaître dans le tableau de bord.
- Des clusters provisionnés manuellement via Terraform restent invisibles dans la plateforme tant qu'ils n'ont pas de fiche en base de données.

Un module de synchronisation (`backend/app/api/sync.py`) a été développé pour résoudre ces deux cas.

Synchronisation (suppression des enregistrements obsolètes). L'endpoint `POST /api/sync` parcourt tous les enregistrements VM et Cluster de l'utilisateur en base de données. Pour chaque enregistrement, la fonction `_terraform_still_exists()` vérifie l'existence et la cohérence du fichier `terraform.tfstate` dans le répertoire d'état correspondant :

Listing 3.7. Détection d'une ressource cloud supprimée via l'état Terraform

```

1 def _terraform_still_exists(state_dir: Path) -> bool:
2     tfstate = state_dir / "terraform.tfstate"
3     if not tfstate.exists():
4         return False
5     resources = json.loads(tfstate.read_text())["resources"]
6     live = sum(len(r.get("instances", [])) for r in resources)
7     return live > 0

```

Si aucune instance vivante n'est trouvée, l'enregistrement est supprimé de la base de données.

Import (découverte de ressources non enregistrées). L'endpoint POST `/api/sync/import` parcourt les répertoires `terraform_states/` (VM) et `terraform_cluster_states/` (clusters). Pour chaque répertoire dont l'état Terraform est actif mais qui n'a pas d'enregistrement en base de données, un enregistrement de substitution est créé en lisant les variables `terraform.auto.tfvars.json` et les outputs du fichier `terraform.tfstate`.

Ces deux fonctionnalités sont accessibles depuis le tableau de bord via les boutons **Sync** et **Importer depuis le cloud**.

[Screenshot : Tableau de bord — boutons Sync et Import avec notifications de résultat]

3.5 Conclusion

Trois sprints itératifs ont livré une plateforme cloud en libre-service complète : le Sprint 1 a établi le cur du provisionnement de VM avec paiement et suivi SSE ; le Sprint 2 a ajouté la recommandation IA, l'accès SSH et la supervision Netdata ; le Sprint 3 a résolu le provisionnement de clusters k3s dans les contraintes de quota EIP de l'environnement partagé MESRS, y ajoutant le terminal SSH pour clusters, un contrôleur d'autoguérison hub-spoke et des outils de réconciliation de l'état cloud. Le schéma maître-comme-passerelle-NAT avec `source_dest_check=false` au niveau Terraform, et le choix de k3s pour sa simplicité d'installation dans un réseau contraint, constituent les décisions architecturales clés du sprint. Le chapitre suivant valide la plateforme au regard de ses objectifs initiaux.

Chapitre 4 Tests et Validation

4.1 Introduction

Ce chapitre rend compte de la validation de VM Marketplace au regard des objectifs définis dans l'introduction générale. Tous les tests ont été réalisés sur l'infrastructure HCS MESRS de production (*mesrscloud.rnu.tn*, région `tn-global-1`). En l'absence de suite de tests automatisés, la validation suit un protocole de tests manuels structuré par module fonctionnel, avec le provisionnement cloud réel comme principale preuve de validation.

4.2 Environnement de Test

TABLE 4.1. Environnement de test

Composant	Configuration
Région HCS	<code>tn-global-1</code> , projet <code>ñ Marketplace z</code>
Réseau	VPC avec sous-réseau <code>47a8cd69-519b-45fe-845b-23704f01ace6</code>
Images OS testées	Ubuntu 22.04 LTS (UUID HCS)
Flavor maître	<code>c7n.xlarge.4</code> (4 vCPU, 16 Go RAM)
Flavor worker	<code>c7n.xlarge.4</code> (4 vCPU, 16 Go RAM)
Disque système	50 Go SSD
Distribution K8s provisionnée	<code>k3s v1.31.4+k3s1</code>
Navigateur de test	Chromium / Firefox
Hôte backend	<code>localhost:8000</code> (Python 3.11, FastAPI)
Hôte frontend	<code>localhost:3000</code> (Next.js 14)
Moteur IA	<code>localhost:8001</code> , Ollama llama3.1 (8B)

4.3 Tests d'Authentification

TABLE 4.2. Résultats des tests d'authentification

TC	Cas de test	Attendu	Résultat
TC-A01	Inscription avec e-mail et mot de passe valides	201, utilisateur stocké en BD	✓
TC-A02	Inscription avec e-mail déjà existant	Erreur 400 retournée	✓
TC-A03	Connexion avec identifiants valides	200, jeton JWT retourné	✓
TC-A04	Connexion avec mauvais mot de passe	401 Non autorisé	✓
TC-A05	Accès à un endpoint protégé sans jeton	401 Non autorisé	✓
TC-A06	Accès à un endpoint protégé avec jeton expiré	401 Non autorisé	✓

4.4 Tests de Provisionnement de VM

TABLE 4.3. Résultats des tests de provisionnement de VM

TC	Cas de test	Attendu	Résultat
TC-V01	Compléter l'assistant avec config valide + paiement	Session ID retournée	✓
TC-V02	Le flux de progression SSE s'ouvre et envoie les étapes	6 étapes diffusées	✓
TC-V03	La VM apparaît dans la console HCS après apply	Instance ECS en état Running	✓
TC-V04	IP publique de la VM accessible via SSH	Connexion root par clé SSH réussie	✓
TC-V05	E-mail de confirmation reçu	E-mail avec détails VM livré	✓
TC-V06	Tableau de bord Netdata accessible sur port 19999	Interface Netdata chargée avec métriques	✓
TC-V07	La VM apparaît dans le tableau de bord	Carte VM visible avec IP correcte	✓
TC-V08	Supprimer une VM depuis le tableau de bord	terraform destroy exécuté ; VM retirée de la BD	✓
TC-V09	Carte Stripe invalide rejetée	Erreur 400, aucun provisionnement lancé	✓

[Screenshot : Console HCS ECS affichant une instance VM provisionnée en état Running]

4.5 Tests de Recommandation IA

TABLE 4.4. Résultats des tests de recommandation IA

TC	Cas de test	Attendu	Résultat
TC-IA01	Décrire une charge de travail d'entraînement ML	Flavor haut niveau recommandé	✓
TC-IA02	Décrire un serveur web statique	Flavor bas niveau (1–2 vCPU)	✓
TC-IA03	Description vague (ex. : n quelque chose de rapide z)	Score de confiance faible ($< 0,5$)	✓
TC-IA04	Moteur IA hors ligne	Erreur 503 retournée par le backend	✓
TC-IA05	Recommandation pré-remplit l'assistant	Assistant ouvert avec les specs correctes	✓

[Screenshot : Résultat de recommandation IA — analyse de charge de travail et flavor recommandé pour un entraînement de machine learning]

4.6 Tests du Terminal SSH

TABLE 4.5. Résultats des tests du terminal SSH

TC	Cas de test	Attendu	Résultat
TC-S01	Ouvrir le terminal SSH pour une VM en cours d'exécution	xterm.js connecté, invite shell affichée	✓
TC-S02	Exécuter des commandes (ls, top, apt) dans le terminal	Commandes exécutées, sortie rendue	✓
TC-S03	Terminal pour VM sans IP publique	Erreur d'affichée gracieusement	✓
TC-S04	Fermer l'onglet du navigateur	WebSocket et canal SSH fermés côté serveur	✓

[Screenshot : Terminal SSH dans le navigateur affichant une session shell root interactive sur une VM HCS]

4.7 Tests du Cluster k3s

TABLE 4.6. Résultats des tests du cluster k3s

TC	Cas de test	Attendu	Résultat
TC-K01	Compléter l'assistant cluster + paiement	Session ID retournée	✓
TC-K02	Flux de progression SSE	Toutes les étapes atteignent l'état final	✓
TC-K03	Instance maître ECS créée sur HCS	ECS en état Running avec EIP	✓
TC-K04	Instance worker ECS créée (sans EIP)	ECS en état Running, IP privée uniquement	✓
TC-K05	Le worker accède à Internet via le NAT maître	Installation k3s agent réussie sur le worker	✓
TC-K06	kubectl get nodes affiche tous les nœuds Ready	STATUS : Ready pour maître et workers	✓
TC-K07	kubectl cluster-info affiche l'URL du serveur API	Serveur API à l'EIP maître:6443	✓
TC-K08	Déployer nginx et passer à 3 réplicas	Tous les pods Running, service accessible	✓
TC-K09	Télécharger le kubeconfig depuis le tableau de bord	Fichier YAML valide ; kubectl se connecte	✓
TC-K10	Supprimer le cluster depuis le tableau de bord	terraform destroy terminé ; entrée BD supprimée	✓
TC-K11	Terminal SSH navigateur vers le nœud maître	Shell interactif ouvert dans xterm.js	✓

Sortie de validation kubectl. La sortie suivante a été obtenue après téléchargement du kubeconfig et exécution de kubectl sur le cluster provisionné :

Listing 4.1. Sortie de kubectl get nodes sur le cluster k3s provisionné

```
1 $ kubectl get nodes -o wide
2 NAME                                STATUS    ROLES                                AGE
   VERSION
3 k3s-cluster-master                 Ready    control-plane,master               8m
   v1.31.4+k3s1
4 k3s-cluster-worker-1              Ready    <none>                             5m
   v1.31.4+k3s1
```

Listing 4.2. Sortie de kubectl cluster-info

```
1 $ kubectl cluster-info
2 Kubernetes control plane is running at https
   ://193.95.30.249:6443
3 CoreDNS is running at https://193.95.30.249:6443/api/v1/
   namespaces/...
```

[Screenshot : kubectl get nodes — les deux nuds k3s en statut Ready avec la version v1.31.4+k3s1]

4.8 Tests d'Autoguérison et de Synchronisation

TABLE 4.7. Résultats des tests d'autoguérison et de synchronisation

TC	Cas de test	Attendu	Résultat
TC-SH01	Activer l'autoguérison dans l'assistant + paiement	Option incluse dans le total (\$15/mois)	✓
TC-SH02	Enregistrement spoke auprès du hub après provisionnement	Statut registered en BD	✓
TC-SH03	Cluster provisionné sans autoguérison	Champ enable_self_healing = false	✓
TC-SY01	Sync : VM supprimée depuis la console HCS	Enregistrement VM supprimé de la BD	✓
TC-SY02	Sync : cluster supprimé hors plateforme	Enregistrement cluster supprimé de la BD	✓
TC-SY03	Import : répertoire Terraform sans fiche BD	Enregistrement stub créé avec IP et config	✓
TC-SY04	Import : répertoire Terraform avec état vide	Aucun enregistrement créé	✓

4.9 Évaluation des Objectifs

Le tableau 4.8 évalue chacun des cinq objectifs initiaux au regard des preuves de validation.

TABLE 4.8. Évaluation des objectifs

Objectif	Pré-Statut
Application web full-stack avec assistant VM, Stripe et Terraform sur HCS	TC✓Atteint V01 à TC- V08 va- li- dés ; VMs réelles pro- vi- sion- nées sur HCS de pro- duc- tion
Moteur de recommandation IA (Llama 3.1, local)	TC✓Atteint IA01 à TC- IA05 va- li- dés ; re- com- man- da- tions gé- né- rées en moins de 30 s
Terminal SSH navigateur et supervision Netdata	42 / 53 TC✓Atteint S01 s

4.10 Limites Identifiées

Les limites suivantes ont été identifiées et documentées dans le cadre d’une évaluation honnête :

1. **Suivi de progression en mémoire.** Les dictionnaires `progress_store` et `cluster_progress` sont stockés en mémoire de processus. Un redémarrage du backend pendant le provisionnement (qui dure 5 à 10 minutes) orphelinise le flux de progression ; la VM ou le cluster sera tout de même créé sur HCS mais le frontend n’affichera plus de mises à jour.
2. **Absence de suite de tests automatisés.** Le projet ne dispose pas de tests unitaires, d’intégration ni de pipeline CI. Toute la validation a été effectuée manuellement sur l’environnement HCS de production.
3. **SQLite pour la persistance.** SQLite est adapté à un déploiement de développement à instance unique, mais ne supporte pas les écritures simultanées de plusieurs workers backend. Une migration vers PostgreSQL serait nécessaire pour une mise à l’échelle en production.
4. **URLs localhost codées en dur.** Les variables `NEXT_PUBLIC_API_URL` et les URLs du moteur IA sont codées en dur vers localhost. Les déploiements multi-hôtes ou containerisés nécessitent une configuration par variable d’environnement.
5. **Absence de contrôle d’accès par rôle.** Il n’existe pas de RBAC (contrôle d’accès basé sur les rôles) ni de gestion de quota par utilisateur au niveau de la plateforme. Tout utilisateur authentifié peut provisionner autant de ressources que le quota du projet HCS le permet.
6. **Sécurité du kubeconfig.** Le kubeconfig stocké en base de données accorde un accès cluster-admin. Il est transmis via HTTPS mais stocké en clair dans la base de données SQLite.

4.11 Conclusion

Les cinq objectifs initiaux ont tous été atteints et validés sur l’infrastructure HCS MESRS de production. La plateforme provisionne avec succès aussi bien des VM individuelles que des clusters Kubernetes multi-nuds, offre une recommandation assistée par IA, et propose un accès SSH intégré ainsi qu’une supervision automatisée. Les limites identifiées sont bien comprises et constituent une feuille de route claire pour une évolution vers une version de production.

Conclusion Générale

Synthèse de la Démarche

Ce projet a cherché à répondre à une problématique concrète : comment permettre aux chercheurs et aux étudiants des institutions académiques tunisiennes de provisionner et gérer de façon autonome des machines virtuelles et des clusters Kubernetes sur l'infrastructure Huawei Cloud Stack (HCS) du MESRS, sans dépendre d'une intervention manuelle d'un administrateur ?

La réponse apportée est **VM Marketplace**, une plateforme cloud full-stack en libre-service développée sur six mois de stage chez Huawei Technologies Tunisie. En travaillant de manière itérative à travers trois sprints, nous avons bâti une architecture à trois services — un frontend Next.js, un backend FastAPI et un moteur d'inférence IA distinct — qui intègre le paiement Stripe, le provisionnement d'infrastructure IaC via Terraform sur HCS, la diffusion SSE pour le suivi en temps réel, des recommandations de VM assistées par IA via un modèle Llama 3.1 hébergé localement, un accès SSH dans le navigateur, la supervision Netdata, un gestionnaire complet du cycle de vie des clusters k3s, un contrôleur d'autoguérison hub-spoke et des outils de réconciliation entre la base de données et l'état Terraform cloud.

Principaux Résultats

- Le provisionnement bout en bout de VM sur l'environnement HCS de production a été validé, avec des VM atteignant l'état Running et la supervision Netdata installée automatiquement via SSH.
- Un cluster k3s à deux nuds (v1.31.4+k3s1) a été provisionné avec succès, les deux nuds atteignant l'état Ready. L'architecture réseau maître-comme-passerelle-NAT — combinant `source_dest_check=false` au niveau Terraform avec des règles iptables MASQUERADE Linux — a résolu la contrainte de quota EIP sans nécessiter de ressources cloud supplémentaires. Le choix de k3s a simplifié l'installation à une seule commande par nud, supprimant les dépendances apt problématiques dans un réseau contraint.
- Le contrôleur d'autoguérison hub-spoke a été validé : l'enregistrement du spoke auprès du hub s'effectue automatiquement après provisionnement, avec propagation du statut en base de données.
- Les outils de synchronisation et d'import ont résolu la divergence entre l'état Terraform réel et les enregistrements de la base de données.
- Le moteur de recommandation IA, fonctionnant entièrement sur une instance Ollama

locale, a correctement mappé des descriptions textuelles de charges de travail vers des recommandations de flavor HCS avec score de confiance.

- Les cinq objectifs initiaux ont été atteints et validés sur l'infrastructure de production.

Difficultés et Enseignements

Le défi technique le plus significatif de ce projet a été la découverte et la résolution du comportement `source_dest_check` au niveau de l'hyperviseur HCS. Cette contrainte n'est pas documentée dans la documentation standard du fournisseur Terraform `huaweicloud/hcs` et n'a été identifiée qu'après une élimination systématique des autres causes possibles (règles iptables, paramètres noyau, DNS). L'enseignement retenu est que les comportements spécifiques à l'hyperviseur cloud requièrent une investigation empirique et ne peuvent pas être supposés conformes aux comportements génériques de la mise en réseau Linux.

Sur le plan du génie logiciel, ce projet a renforcé la valeur d'une approche itérative lorsqu'on travaille dans un environnement à inconnues externes : chaque sprint a validé un ensemble d'hypothèses sur l'environnement HCS et a adapté la conception en conséquence.

Compétences Acquisées

Ce projet m'a permis de construire et de consolider les compétences suivantes :

- **Infrastructure cloud** : gestion des ressources Huawei Cloud Stack, Terraform IaC, configuration réseau VPC, gestion des EIP et configuration cloud-init.
- **Administration Kubernetes** : provisionnement de clusters k3s, réseau Flannel intégré, opérations kubectl, gestion du kubeconfig, modèle hub-spoke pour l'autoguérison.
- **Développement backend** : API REST FastAPI, Server-Sent Events, authentification JWT, gestion de threads en arrière-plan, automatisation SSH Paramiko.
- **Intégration IA** : ingénierie de prompts LLM avec sortie JSON structurée, hébergement de modèle local avec Ollama, systèmes de recommandation à score de confiance.
- **Intégration full-stack** : coordination de trois services indépendants (frontend, backend, moteur IA) avec un contrat d'API partagé.

Apport pour l'Organisme d'Accueil

VM Marketplace fournit à Huawei Technologies Tunisie un prototype fonctionnel de portail cloud en libre-service adapté aux spécificités de l'environnement HCS MESRS. La solution réseau pour les nuds workers Kubernetes privés documentée dans ce rapport (`source_dest_check` + iptables MASQUERADE) constitue une connaissance technique réutilisable pour les futurs déploiements HCS. La plateforme démontre également qu'une couche de recommandation IA basée sur un LLM hébergé localement est viable sans dépendre

d'API IA cloud externes, ce qui est particulièrement pertinent dans un contexte de données sensibles de recherche.

Perspectives et Travaux Futurs

Quatre améliorations concrètes sont proposées pour une évolution vers une version de production :

1. **Persistance du suivi de progression** : remplacer les dictionnaires `progress_store` en mémoire par un magasin Redis ou une table journal en base de données afin de survivre aux redémarrages du backend pendant les opérations de provisionnement longues.
2. **Suite de tests automatisés** : implémenter des tests unitaires pour la couche service (en simulant les appels de sous-processus Terraform et les connexions SSH) et des tests d'intégration contre un projet HCS de staging.
3. **RBAC multi-tenant** : ajouter des rôles utilisateur (administrateur, utilisateur standard) et des quotas de ressources par utilisateur appliqués au niveau de la plateforme, indépendamment des quotas du projet HCS.
4. **Migration PostgreSQL et containerisation** : remplacer SQLite par PostgreSQL et containeriser les trois services avec Docker Compose ou un chart Helm Kubernetes, permettant la mise à l'échelle horizontale et le déploiement multi-hôtes.

Réflexion Finale

Construire une plateforme qui réduit l'écart entre un environnement cloud de production et des utilisateurs académiques non experts a nécessité de naviguer en permanence entre la conception de l'expérience utilisateur, l'ingénierie des systèmes distribués et les contraintes techniques spécifiques de l'infrastructure HCS. Ce stage a démontré que combler ces couches — rendre les ressources cloud puissantes accessibles aux chercheurs qui souhaitent simplement exécuter leur code — est à la fois techniquement exigeant et véritablement utile.

Bibliographie / Netographie

- [1] HUAWEI TECHNOLOGIES CO., LTD., *2023 Annual Report*, <https://www.huawei.com/en/annual-report/2023>, Accessed : 2026-06-01, 2024.
- [2] MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA RECHERCHE SCIENTIFIQUE, *MESRS Cloud Platform — Huawei Cloud Stack for Academic Institutions*, <https://mesrscloud.rnu.tn>, Internal platform ; Accessed : 2026-06-01.
- [3] HUAWEI CLOUD, *Huawei Cloud Stack Documentation*, <https://support.huaweicloud.com/intl/en-us/hcs/index.html>, Accessed : 2026-06-01.
- [4] HUAWEI CLOUD, *Terraform Provider for Huawei Cloud Stack (huaweicloud/hcs)*, v2.4.0, <https://registry.terraform.io/providers/huaweicloud/hcs/latest/docs>, Accessed : 2026-06-01.
- [5] OPENSTACK FOUNDATION, *OpenStack Horizon — The OpenStack Dashboard*, <https://docs.openstack.org/horizon>, Accessed : 2026-06-01.
- [6] PROXMOX SERVER SOLUTIONS GMBH, *Proxmox VE Documentation*, <https://pve.proxmox.com/pve-docs>, Accessed : 2026-06-01.
- [7] VMWARE, INC., *VMware Cloud Director Documentation*, <https://docs.vmware.com/en/VMware-Cloud-Director>, Accessed : 2026-06-01.
- [8] AMAZON WEB SERVICES, *AWS Management Console — Overview*, <https://aws.amazon.com/console>, Accessed : 2026-06-01.
- [9] VERCEL, *Next.js Documentation*, v14, <https://nextjs.org/docs>, Accessed : 2026-06-01.
- [10] TAILWIND LABS, *Tailwind CSS Documentation*, <https://tailwindcss.com/docs>, Accessed : 2026-06-01.
- [11] S. RAMÍREZ, *FastAPI Documentation*, <https://fastapi.tiangolo.com>, Accessed : 2026-06-01.
- [12] OLLAMA, *Ollama — Run Large Language Models Locally*, <https://ollama.com/docs>, Accessed : 2026-06-01.
- [13] META AI, *Llama 3.1 — Open Foundation and Fine-Tuned Chat Models*, <https://ai.meta.com/blog/meta-llama-3-1>, Accessed : 2026-06-01, 2024.
- [14] PARAMIKO CONTRIBUTORS, *Paramiko Documentation*, <https://www.paramiko.org>, Accessed : 2026-06-01.
- [15] NETDATA INC., *Netdata Documentation*, <https://learn.netdata.cloud>, Accessed : 2026-06-01.

- [16] RANCHER LABS (SUSE), *k3s — Lightweight Kubernetes*, <https://k3s.io>, Accessed : 2026-06-15.

Chapitre A Référence des Endpoints REST

Cette annexe liste les principaux endpoints REST exposés par le backend FastAPI. Tous les endpoints protégés nécessitent un en-tête `Authorization: Bearer <jeton>`.

Méthode Description	Chemin
POST Inscrire un nouvel utilisateur	/api/register
POST Authentifier et recevoir un JWT	/api/login
GET Retourner l'utilisateur authentifié courant	/api/users/me
POST Confirmer le paiement Stripe, lancer le provisionnement de VM	/api/payment/create-payment-intent
GET Flux SSE de progression du provisionnement	/api/payment/progress/:session_id
GET Lister toutes les VM de l'utilisateur courant	/api/vms/
DELETE Supprimer une VM (exécute <code>terraform destroy</code>)	/api/vms/:vm_id
GET Terminal SSH WebSocket pour une VM	/api/ssh/terminal/:vm_id
GET Retourner l'URL Netdata d'une VM	/api/monitoring/:vm_id
POST Proxy vers le moteur IA ; retourne la recommandation de VM	/api/ai/recommend
GET Vérification de l'état du moteur IA	/api/ai/health
POST Confirmer le paiement Stripe, lancer le provisionnement du cluster K8s	/api/clusters/pay-and-create
GET Flux SSE de progression du cluster	/api/clusters/progress/:session_id
GET Lister tous les clusters de l'utilisateur courant	/api/clusters/
GET Retourner le kubeconfig d'un cluster	/api/clusters/:cluster_id/kubeconfig

Méthode	Chemin
Description	
Télécharger le kubeconfig YAML	
DELETE	/api/clusters/:cluster_id
Supprimer un cluster (exécute terraform destroy)	

TABLE A.1. Endpoints REST de VM Marketplace

Chapitre B Modèle Terraform du Cluster

Cette annexe contient le modèle Terraform complet utilisé pour le provisionnement des clusters Kubernetes (terraform_cluster/main.tf).

Listing B.1. terraform_cluster/main.tf — Infrastructure du cluster K8s

```
1 terraform {
2   required_providers {
3     hcs = {
4       source = "huaweicloud/hcs"
5       version = "2.4.0"
6     }
7   }
8 }
9
10 provider "hcs" {
11   region          = "tn-global-1"
12   project_name    = "Marketplace"
13   cloud           = "mesrscloud.rnu.tn"
14   access_key      = var.access_key
15   secret_key      = var.secret_key
16   insecure        = true
17 }
18
19 data "hcs_availability_zones" "zones" {}
20 data "hcs_ecs_compute_flavors" "all" {}
21
22 locals {
23   selected_az = data.hcs_availability_zones.zones.names[0]
24
25   master_flavors = [
26     for f in data.hcs_ecs_compute_flavors.all.flavors : f
27     if f.id == var.master_flavor_id || f.name == var.
28       master_flavor_id
29   ]
30   resolved_master_flavor_id = length(local.master_flavors) > 0
31     ? local.master_flavors[0].id : var.master_flavor_id
32
33   worker_flavors = [
34     for f in data.hcs_ecs_compute_flavors.all.flavors : f
35     if f.id == var.worker_flavor_id || f.name == var.
```

```

        worker_flavor_id
35     ]
36     resolved_worker_flavor_id = length(local.worker_flavors) > 0
37         ? local.worker_flavors[0].id : var.worker_flavor_id
38 }
39
40 # Noeud maitre
41 resource "hcs_ecs_compute_instance" "master" {
42     name                = "${var.cluster_name}-master"
43     availability_zone    = local.selected_az
44     flavor_id           = local.resolved_master_flavor_id
45     image_id            = trimspace(var.image_id)
46     delete_eip_on_termination = false
47     security_group_ids   = [var.security_group_id]
48     system_disk_type     = var.system_disk_type
49     system_disk_size     = var.system_disk_size
50
51     network {
52         uuid                = var.subnet_id
53         source_dest_check = false # requis pour le transfert NAT
54     }
55
56     admin_pass = var.administrator_password
57
58     user_data = base64encode(<<-EOT
59         #cloud-config
60         disable_root: false
61         ssh_authorized_keys:
62             - ${var.ssh_public_key}
63     EOT)
64 }
65
66 # EIP du maitre
67 resource "hcs_vpc_eip" "master_eip" {
68     depends_on = [hcs_ecs_compute_instance.master]
69     publicip { type = "External-01" }
70     bandwidth {
71         name      = "bw-${var.cluster_name}-master"
72         size      = 5
73         share_type = "PER"
74     }
75 }
76
77 resource "hcs_vpc_eip_associate" "master_eip_assoc" {
78     public_ip = hcs_vpc_eip.master_eip.address

```

```

79   port_id    = hcs_ecs_compute_instance.master.network[0].port
80 }
81
82 # Noeuds workers (IP privée uniquement -- pas d'EIP)
83 resource "hcs_ecs_compute_instance" "workers" {
84   count          = var.worker_count
85   name           = "${var.cluster_name}-worker-${
86     count.index+1}"
87   availability_zone = local.selected_az
88   flavor_id       = local.resolved_worker_flavor_id
89   image_id        = trimspace(var.image_id)
90   delete_eip_on_termination = false
91   security_group_ids = [var.security_group_id]
92   system_disk_type  = var.system_disk_type
93   system_disk_size  = var.system_disk_size
94
95   network { uuid = var.subnet_id }
96
97   admin_pass = var.administrator_password
98
99   user_data = base64encode(<<-EOT
100     #cloud-config
101     disable_root: false
102     ssh_authorized_keys:
103       - ${var.ssh_public_key}
104   EOT)
105 }
106
107 output "master_public_ip" { value = hcs_vpc_eip.master_eip.
108   address }
109
110 output "master_private_ip" {
111   value = hcs_ecs_compute_instance.master.network[0].
112     fixed_ip_v4
113 }
114
115 output "worker_private_ips" {
116   value = [for w in hcs_ecs_compute_instance.workers
117     : w.network[0].fixed_ip_v4]
118 }

```